

Research-Grade Computational Physics in Python

The Complete TaN-MM-Org Course: Eight Packages, One Discipline

Tanvir Mahmud Mahim

Course edition 1.0, September 2026

Contents

Research-Grade Computational Physics in Python	3
The Complete TaN-MM-Org Course: Eight Packages, One Discipline	3
Part I — Electronic Structure	5
Module 1 — hamop: One Hamiltonian, Every Observable	6
1.0 Why this package exists	6
1.1 Conventions you must internalize first	6
Lesson 1.1 — Your first model: the SSH chain	6
Lesson 1.2 — Graphene and the universal conductivity	8
Lesson 1.3 — Topology: Chern numbers two independent ways	9
Lesson 1.4 — Transport and local spectroscopy	10
1.5 Pitfalls	12
1.6 Exercises	12
1.7 Further reading	12
Module 2 — kpenvelope: Holes in Wurtzite Heterostructures	13
2.0 Why this package exists	13
2.1 Conventions	13
Lesson 2.1 — Hard-wall subbands against the closed form	13
Lesson 2.2 — Real GaN: bulk anchors and a quantum well	15
Lesson 2.3 — Dispersion, masses, and intersubband optics	16
2.4 Pitfalls	18
2.5 Exercises	18
2.6 Further reading	18
Part II — Measurement and Inference	19
Module 3 — ramansep: Strain or Doping? Ask Two Phonons	20
3.0 Why this package exists	20
3.1 Conventions	20
Lesson 3.1 — The inversion, exact round trip, and the paper’s edge charge	20
Lesson 3.2 — From raw spectra to values	22
Lesson 3.3 — More modes, model checking, and your own calibration	23
3.4 Pitfalls	25
3.5 Exercises	25
3.6 Further reading	26
Module 4 — sparq-triage: Is It a Single Photon Source?	27
4.0 Why this package exists	27
4.1 Conventions	27
Lesson 4.1 — The twin and the exact reference	27
Lesson 4.2 — Three honest answers to “what is $g_2(0)$?”	29
Lesson 4.3 — Stop measuring when you know	31
4.4 Pitfalls	32
4.5 Exercises	32
4.6 Further reading	33

Module 5 — absnoise: The Noise Floor of Andreev Thermometry	34
5.0 Why this package exists	34
Lesson 5.1 — Recipes, junctions, and the thermometry bound	34
Lesson 5.2 — Telegraph noise and the full device budget	36
Lesson 5.3 — Fit your own junction from $I_c(T)$	37
5.4 Pitfalls	39
5.5 Exercises	39
5.6 Further reading	39
Part III — Quantum Light and Metrology	40
Module 6 — cavsqueeze: Spin Squeezing You Can Design Against	41
6.0 Why this package exists	41
6.1 Conventions	41
Lesson 6.1 — Twist an ensemble, project a clock	41
Lesson 6.2 — The exact benchmark and the $N^{-2/3}$ law	43
Lesson 6.3 — Squeezing from measured counts	44
6.4 Pitfalls	45
6.5 Exercises	45
6.6 Further reading	46
Module 7 — sqzcomb: Squeezed Light from Kerr Microcombs	47
7.0 Why this package exists	47
7.1 Conventions	47
Lesson 7.1 — Steady states and the DPO anchor	47
Lesson 7.2 — The photonic molecule and the loss budget	49
Lesson 7.3 — Entanglement, intracavity and detected	50
7.4 Pitfalls	52
7.5 Exercises	52
7.6 Further reading	53
Part IV — Closing the Loop with Fabrication	54
Module 8 — fabtwin: Learn the Fab, Then Beat Its Tail	55
8.0 Why this package exists	55
8.1 Conventions	55
Lesson 8.1 — Exact optics and adjoint design	55
Lesson 8.2 — Learn the process, ascend the tail	57
Lesson 8.3 — Bring your own fabrication data	59
8.4 Pitfalls	61
8.5 Exercises	61
8.6 Further reading	61
Capstone Projects	62
Appendix A — Version and Citation Table	63
Appendix B — The Code Bundle	64
Colophon	65

Research-Grade Computational Physics in Python

The Complete TaN-MM-Org Course: Eight Packages, One Discipline

By **Tanvir Mahmud Mahim** — Department of Electrical and Electronic Engineering, BRAC University

Course edition 1.0 — September 2026

About this course

This course teaches the eight open-source scientific Python packages of the TaN-MM-Org organization, written and maintained by the author, from first use to research-grade workflows:

#	Package	Version taught	Domain
1	hamop	0.6.0	Tight-binding electronic structure, optics, topology, quantum transport
2	kpenvelope	0.7.0	Six-band k·p envelope functions for wurtzite heterostructures
3	ramansep	0.7.0	Strain/doping separation in 2D-material Raman maps
4	sparq-triage	0.5.0	Single-photon-emitter characterization and autonomous triage
5	absnoise	0.6.0	Andreev-bound-state noise and Josephson thermal detectors
6	cavsqueeze	1.11.0	Cavity-mediated spin squeezing and quantum metrology
7	sqzcomb	0.8.0	Squeezed light in Kerr microcombs
8	fabtwin	0.2.0	Learned fabrication-process twins and yield-aware inverse design

All eight are installable from PyPI, tested in continuous integration on Python 3.9–3.13 (fabtwin: 3.10–3.13), Apache-2.0 licensed, and archived release-by-release on Zenodo. Each grew out of

a research paper by the author and collaborators and was then generalized into a reusable instrument; every module of this course names its paper and its concept DOI.

Every number in this book is real. Each lesson is a runnable script, shipped in the companion code bundle, and every output shown in these pages was produced by running that exact script against the exact released package versions listed above. Nothing is transcribed from memory, and nothing is idealized: where an optimizer lands within a tolerance rather than exactly, the book says so and shows the tolerance.

Who this course is for

Researchers and graduate students in condensed-matter physics, quantum optics, semiconductor physics, and quantum technology. You should be comfortable with quantum mechanics at the graduate-course level, basic solid-state physics (bands, phonons, second quantization), and Python with NumPy. Each module briefly states the physics it needs, but this is a course on *doing the computations correctly*, not a physics textbook.

The one idea that runs through all eight packages

These packages share a discipline you will meet in every module: **every physical claim is anchored to something exact** — a closed-form result, an independent implementation of the same quantity, or an exact identity — and never to a stored number that a test merely reproduces. When a package cannot do something honestly (a parameter with no citable value, a gradient that would have to be faked, a quantity that is not identifiable from the data), it *refuses with an explanation* rather than guessing. You will see refusals throughout this course, on purpose: learning where a tool says “no” is learning where its answers can be trusted.

This philosophy has a practical payoff for you: the same anchors the test suites use are available to *you* as sanity checks on your own inputs. This course uses them constantly.

Setting up

```
python -m pip install hamop kpenvelope ramansep sparq-triage \
                        absnoise cavsqueue sqzcomb fabtwin
# optional extras used in clearly marked places:
#   pip install 'sparq-triage[ml]'      # PyTorch estimators / RL (Module 4 mentions)
#   pip install 'fabtwin[twin]'        # JAX WGAN-GP twin (Module 8 mentions)
```

The lesson scripts in the code bundle print their package version at the start; if your installed versions are newer than the ones taught here, the numbers may differ in the last digits, and the release notes on each GitHub repository state what changed.

A note on floating point: the outputs reproduced in this book were generated on Linux x86-64 with NumPy 2.x. Bit-level floating-point results can differ across BLAS builds; every assertion in the lesson scripts uses tolerances chosen so they pass on any standard platform.

How each module is organized

Each module follows the same arc: *what the package is and where it comes from* (the research it distills, with citations), *the conventions you must know* (units, orderings, sign conventions — the things that silently ruin calculations), *worked lessons* (complete scripts with their verified outputs), *pitfalls*, and *exercises* with hints. The lessons are cumulative within a module but the modules are independent; read Module 8 first if fabrication is your problem today.

Part I — Electronic Structure

Module 1 — hamop: One Hamiltonian, Every Observable

Package: hamop 0.6.0 — `pip install hamop` — github.com/TaN-MM-Org/hamop — DOI [10.5281/zenodo.22311381](https://doi.org/10.5281/zenodo.22311381)

Methodological basis: the package is the general-purpose engine distilled from the pipeline of “*Learning the quantum Hamiltonian of defective monolayer MoS2 reveals collective vacancy brightness decoupled from defect count*” (code: github.com/Tanvir-Mahmud-Mahim/mos2-vacancy-optics), where the optics, electronic structure and transport of vacancy-disordered MoS2 had to be computed from one Hamiltonian, strictly consistently.

1.0 Why this package exists

Build a tight-binding Hamiltonian once — real-space hopping blocks, orthogonal or LCAO-style nonorthogonal basis — and compute its band structure, density of states, Kubo-Greenwood optical conductivity, Berry-phase topology, and Landauer (NEGF) transmission from the same matrices. The point is the *consistency*, not any single solver: when a model’s optics and its spectrum come from different codes with different velocity gauges, overlap conventions or broadenings, they drift apart, and suddenly an absorption edge no longer sits at the band gap. In hamop every observable diagonalizes the same Bloch assembly, and the Kubo velocity operator is built from the exact k -derivative of that same assembly, so spectral, optical and transport statements about one model cannot disagree.

hamop deliberately does not replace PythTB, pybinding or Kwant, which are more capable in their own cores. Its niche is the combination they leave open: nonorthogonal overlap matrices as first-class citizens across *all* observables, optics/transport/spectrum from one assembly, and a small NumPy/SciPy-only core validated line by line against closed forms.

1.1 Conventions you must internalize first

Energies in eV, positions in Ångström, k in $1/\text{\AA}$, Cartesian. Each directed hopping block is added **once**; the Hermitian partner is implied (adding both double-counts the bond). Optical conductivity is the real sheet conductivity in units of $e^2/(4\hbar)$, with spin degeneracy an explicit factor (default 2). The velocity operator uses the standard atomistic position gauge; intra-atomic dipole blocks can be added via `model.set_dipole` when on-site transitions matter.

Lesson 1.1 — Your first model: the SSH chain

The Su-Schrieffer-Heeger chain is the ideal first model: two sites per cell, alternating bonds, and a gap with the closed form $2|t_1 - t_2|$ that the code has never been told.

```
"""Module 1, Lesson 1.1 -- Your first tight-binding model (hamop).
```

```
Build the SSH chain from scratch, compute its band structure, gap and
density of states, and check the gap against the closed form  $2|t_1 - t_2|$ .
```

```

Requires: pip install hamop (this course was validated on hamop 0.6.0)
"""
import numpy as np

import hamop
from hamop import TightBindingModel, band_edges, bands, dos

print(f"hamop version: {hamop.__version__}")

# --- build the SSH chain by hand -----
# Two sites per cell (positions in Angstrom), one orbital each,
# 1D lattice with a 2.0 A cell. Each directed hopping block is added
# once; the Hermitian partner is implied.
t1, t2 = -1.0, -0.6 # eV: intra- and inter-cell bonds
m = TightBindingModel(positions=[[0.0], [0.7]], norb=1, cell=[[2.0]])
m.add_hop(0, 1, (0,), [[t1]]) # bond inside the cell
m.add_hop(1, 0, (1,), [[t2]]) # bond to the next cell

# --- band structure along the 1D Brillouin zone -----
kpts = np.linspace(0.0, np.pi / 2.0, 101)[: , None] # k in 1/A; BZ edge pi/a
E = bands(m, kpts) # (nk, 2)
print(f"band shape: {E.shape}")
print(f"valence band top : {E[:, 0].max():+.6f} eV")
print(f"conduction band min : {E[:, 1].min():+.6f} eV")

# --- the gap, against the closed form -----
vbm, cbm, gap = band_edges(m, mu=0.0, mesh=2001)
print(f"band_edges gap : {gap:.6f} eV")
print(f"closed form 2|t1-t2|: {2 * abs(t1 - t2):.6f} eV")
assert abs(gap - 2 * abs(t1 - t2)) < 1e-4 # k-grid resolution limited

# --- density of states -----
w = np.linspace(-2.2, 2.2, 221)
rho = dos(m, w, mesh=4001, eta=0.02)
trapz = getattr(np, "trapezoid", getattr(np, "trapz", None)) # numpy 1/2 compat
print(f"DOS integrates to : {trapz(rho, w):.4f} (2 orbitals/cell)")
print(f"DOS inside the gap : {rho[np.abs(w) < 0.3].max():.2e} (~0)")

# --- the same model from the built-in constructor -----
m2 = hamop.ssh(t1=t1, t2=t2)
E2 = bands(m2, kpts)
print(f"hand-built == built-in ssh(): "
      f"{np.allclose(E, E2, atol=1e-12)}")
print("OK: lesson 1.1 complete")

```

Verified output:

```

hamop version: 0.6.0
band shape: (101, 2)
valence band top : -0.400000 eV
conduction band min : +0.400000 eV
band_edges gap : 0.800004 eV
closed form 2|t1-t2|: 0.800000 eV

```

```
DOS integrates to    : 2.0000 (2 orbitals/cell)
DOS inside the gap   : 1.52e-06 (~0)
hand-built == built-in ssh(): True
OK: lesson 1.1 complete
```

Read the output like a professional. The valence maximum and conduction minimum are at ± 0.4 eV — half the closed-form gap each side of zero, as chiral symmetry demands. The band_edges gap is 0.800004 eV against the exact 0.8: the $4 \cdot 10^{-6}$ discrepancy is the k-grid resolution of the mesh, not physics, and the assertion tolerance says so explicitly. The DOS integrates to exactly the orbital count and vanishes in the gap. Finally the hand-built model agrees with the library's ssh() constructor to 10^{-12} — you now know you can trust your own model-building.

Lesson 1.2 — Graphene and the universal conductivity

Graphene's interband sheet conductivity is *universal*: $\sigma = e^2/4\hbar$, independent of hopping and lattice constant (Kuzmenko et al., PRL 100, 117401 (2008)). Since hamop reports conductivity in exactly those units, the plateau must come out at 1 — this is the package's absolute unit anchor, and your calibration check on any installation.

```
"""Module 1, Lesson 1.2 -- Graphene: Dirac bands and the universal
optical conductivity (hamop).
```

```
The interband conductivity of graphene is universal: sigma = e^2/(4 hbar),
independent of the hopping and the lattice constant (Kuzmenko et al.,
Phys. Rev. Lett. 100, 117401 (2008)). hamop reports sheet conductivity
in exactly those units, so the plateau must come out at ~1. This lesson
also verifies the Dirac closure at K and the bandwidth +/-3|t| at Gamma.
"""
```

```
import numpy as np

import hamop
from hamop import bands, graphene, sigma_optical

t, a = -2.7, 2.46                                # eV, Angstrom
g = graphene(t=t, a=a)

# --- exact band anchors -----
# K point of the hexagonal lattice: |K| = 4 pi / (3 a)
K = np.array([[4 * np.pi / (3 * a), 0.0]])
Gamma = np.array([[0.0, 0.0]])
EK = bands(g, K)[0]
EG = bands(g, Gamma)[0]
print(f"gap at K      : {EK[1] - EK[0]:.3e} eV (Dirac point: 0)")
print(f"bands at Gamma: {EG[0]:+.4f}, {EG[1]:+.4f} eV (exact +/-3|t| "
      f"= +/-{3 * abs(t):.1f})")
assert abs(EK[1] - EK[0]) < 1e-10
assert np.allclose(EG, [-3 * abs(t), 3 * abs(t)], atol=1e-10)

# --- universal interband plateau -----
omega = np.linspace(0.8, 1.8, 6)                # eV, well inside the plateau
sig = sigma_optical(g, omega, mu=0.0, mesh=150, eta=0.12)
print("omega (eV) | sigma / (e^2 / 4 hbar)")
for w, s in zip(omega, sig):
```

```

    print(f"    {w:5.2f}    |    {s:.3f}")
print(f"plateau mean: {sig.mean():.3f} (universal value: 1)")
assert abs(sig.mean() - 1.0) < 0.05
print("OK: lesson 1.2 complete")

```

Verified output:

```

gap at K      : 2.665e-15 eV   (Dirac point: 0)
bands at Gamma: -8.1000, +8.1000 eV (exact +/-3|t| = +/-8.1)
omega (eV) | sigma / (e^2 / 4 hbar)
    0.80 | 1.011
    1.00 | 1.016
    1.20 | 1.023
    1.40 | 1.031
    1.60 | 1.041
    1.80 | 1.053
plateau mean: 1.029 (universal value: 1)
OK: lesson 1.2 complete

```

The Dirac closure at K is exact to 10^{-15} ; the Γ -point bands sit at $\pm 3|t|$ exactly. The plateau values drift a few percent above 1 toward higher ω — that is the finite Gaussian broadening $\eta = 0.12$ eV and the beginning of the van Hove tail, both of which shrink as you refine mesh and eta. A 3% plateau at this mesh is the *expected* accuracy, and knowing that is what lets you budget a production calculation.

Lesson 1.3 — Topology: Chern numbers two independent ways

The Haldane model is the canonical Chern insulator. hamop computes its Chern number from the gauge-invariant lattice field strength (Fukui-Hatsugai-Suzuki, J. Phys. Soc. Jpn. 74, 1674 (2005)) — and, completely independently, from the DC limit of the Kubo Hall conductivity. TKNN quantization (Thouless et al., PRL 49, 405 (1982)) says the two must agree, sign included. Two disjoint code paths, one integer.

```

"""Module 1, Lesson 1.3 -- Topology: the Haldane model's Chern number
and the TKNN quantization of the Hall conductivity (hamop).

```

```

The Haldane model (Phys. Rev. Lett. 61, 2015 (1988)) is the canonical
Chern insulator. hamop computes the Chern number by the gauge-invariant
lattice field strength (Fukui-Hatsugai-Suzuki) and, completely
independently, the DC Hall conductivity from the Kubo tensor -- TKNN
says the two must agree: sigma_xy(0) = C e^2/h.
"""

```

```
import numpy as np
```

```
import hamop
```

```
from hamop import band_edges, chern_number, haldane, sigma_tensor
```

```

# --- topological phase -----
m_top = haldane(t1=-1.0, t2=0.1, phi=np.pi / 2, m_ab=0.0)
C = chern_number(m_top, mesh=24, n_occ=1)
print(f"Haldane (phi=pi/2, m_AB=0) : C = {C:+.6f}")
assert abs(C - round(C)) < 1e-9 and round(C) in (-1, 1)

```

```

# --- trivial phase: large sublattice mass -----
m_triv = haldane(t1=-1.0, t2=0.1, phi=np.pi / 2, m_ab=1.0)
C0 = chern_number(m_triv, mesh=24, n_occ=1)
print(f"Haldane (m_AB = 1.0, trivial) : C = {C0:+.6f}")
assert abs(C0) < 1e-9

# --- TKNN: sigma_xy(0) = C e^2/h, computed via the Kubo tensor -----
vbm, cbm, gap = band_edges(m_top, mu=0.0, mesh=41)
mu = 0.5 * (vbm + cbm) # chemical potential in the gap
print(f"topological gap : {gap:.4f} eV")
# sigma_tensor returns sigma_xy(omega) in units of e^2/(4 hbar);
# TKNN in those units reads sigma_xy(0) = 2 C / pi.
sig = sigma_tensor(m_top, np.array([0.0]), mu=mu, directions=(0, 1),
                  mesh=120, eta=1e-3, spin=1)
C_from_kubo = float(np.pi / 2.0 * np.real(sig[0]))
print(f"sigma_xy(0) route -> C : {C_from_kubo:+.4f}")
print(f"lattice field strength -> C : {C:+.4f}")
assert abs(C_from_kubo - C) < 1e-2 # two independent routes agree
print("OK: lesson 1.3 complete")

```

Verified output:

```

Haldane (phi=pi/2, m_AB=0) : C = +1.000000
Haldane (m_AB = 1.0, trivial) : C = +0.000000
topological gap : 1.0510 eV
sigma_xy(0) route -> C : +1.0000
lattice field strength -> C : +1.0000
OK: lesson 1.3 complete

```

$C = +1.000000$ in the topological phase, exactly 0 in the trivial phase, and the Kubo route lands on +1.0000. When your own model gives a Chern number, run both routes; if they disagree, your mesh or your gap has a problem — not the theorem.

Lesson 1.4 — Transport and local spectroscopy

A chain with one impurity has the exact transmission $T(E) = (4t^2 - E^2) / ((4t^2 - E^2) + \epsilon^2)$. The v0.6 local-spectroscopy tools then go inside the device: the site-resolved LDOS, and the Paulsson-Brandbyge bond-current map (PRB 76, 115117 (2007)), where every transverse cut must carry exactly $T(E)$ — Kirchhoff’s law as a computable check.

```

"""Module 1, Lesson 1.4 -- Quantum transport: Landauer transmission,
local density of states and bond currents (hamop).

```

```

A single-orbital chain with one impurity has the exact transmission
 $T(E) = (4t^2 - E^2) / ((4t^2 - E^2) + \epsilon^2)$ . We check hamop's NEGF
sweep against it, then use the v0.6 local-spectroscopy tools: the
device LDOS and the Paulsson-Brandbyge bond-current map, whose every
transverse cut must carry exactly  $T(E)$  (Kirchhoff for currents).
"""

```

```

import numpy as np

```

```

from hamop import (bond_currents, chain_lead_blocks, device_ldos,
                  transmission)

```

```

t = -1.0                                # lead and device hopping (eV)
eps = 0.8                               # impurity on-site energy (eV)
H00, H01 = chain_lead_blocks(t=t)       # semi-infinite lead blocks

# device: five single-site layers, impurity in the middle
layers_H = [np.array([[0.0]]), np.array([[0.0]]), np.array([[eps]]),
            np.array([[0.0]]), np.array([[0.0]])]
coup_H = [np.array([[t]])] * 4          # couplings between layers

# --- transmission against the closed form -----
E_list = np.array([-1.5, -0.7, 0.0, 0.9, 1.6])
T = transmission(E_list, layers_H, coup_H, H00, H01)
T_exact = (4 * t**2 - E_list**2) / ((4 * t**2 - E_list**2) + eps**2)
print(" E (eV) | T(E) hamop | T(E) exact")
for E, a, b in zip(E_list, T, T_exact):
    print(f" {E:+5.2f} | {a:.6f} | {b:.6f}")
assert np.allclose(T, T_exact, atol=1e-4) # finite-eta limited

# --- local density of states -----
E_probe = [0.0]
ldos = device_ldos(E_probe, layers_H, coup_H, H00, H01)[0]
print("site-resolved LDOS at E = 0 (1/eV):",
      np.array2string(ldos, precision=4))
print("(the impurity site carries less weight than its neighbors)")
assert ldos[2] < ldos[1]                # weight pushed off the impurity

# --- bond currents: every cut carries exactly T(E) -----
E0 = 0.5
J, offs = bond_currents(E0, layers_H, coup_H, H00, H01)
T0 = float(transmission(np.array([E0]), layers_H, coup_H, H00, H01)[0])
print(f"\nT({E0}) = {T0:.6f}; current through each inter-layer cut:")
for cut in range(4):
    left = range(0, cut + 1)
    right = range(cut + 1, 5)
    Jcut = sum(J[i, j] for i in left for j in right)
    print(f" cut {cut}-{cut + 1}: {Jcut:.6f}")
    assert abs(Jcut - T0) < 1e-4        # Kirchhoff: all cuts equal T (eta-limited)

# antisymmetry of the current map
assert np.allclose(J, -J.T, atol=1e-12)
print("OK: lesson 1.4 complete")

```

Verified output:

```

E (eV) | T(E) hamop | T(E) exact
-1.50 | 0.732211 | 0.732218
-0.70 | 0.845778 | 0.845783
+0.00 | 0.862057 | 0.862069
+0.90 | 0.832894 | 0.832898
+1.60 | 0.692303 | 0.692308
site-resolved LDOS at E = 0 (1/eV): [0.1372 0.1811 0.1372 0.1811 0.1372]
(the impurity site carries less weight than its neighbors)

T(0.5) = 0.854210; current through each inter-layer cut:

```

```
cut 0-1: 0.854214
cut 1-2: 0.854214
cut 2-3: 0.854214
cut 3-4: 0.854214
OK: lesson 1.4 complete
```

The transmissions match the closed form to $\sim 10^{-5}$ (the finite $\eta = 10^{-6}$ regularization; tighten η and they tighten). The LDOS shows the impurity site carrying less weight than its neighbors. And the bond-current map delivers the module's most satisfying line: all four cuts carry 0.854214, identically — current conservation to the digit shown.

1.5 Pitfalls

Adding both directed hopping blocks double-counts a bond — add each once. The optical conductivity excludes the intraband (Drude) part; for metals combine `sigma_tensor` with `drude_weight`. `bond_currents` refuses nonorthogonal bases, deliberately: the overlap bond-current partition needs energy-dependent bond operators it does not implement, and a wrong current map is worse than none. Point-group folding (`symmetry_fold`) verifies the symmetry on your Hamiltonian before folding — if it refuses, your model genuinely lacks the symmetry.

1.6 Exercises

1.1 Compute the Zak phase of both SSH dimerizations with `berry_phase`. The individual values are convention-dependent; verify that the *difference* is exactly π — the convention-free statement.

1.2 Reproduce $T(E)$ for a two-impurity chain and look for the resonance between the impurities. Then attach a Büttiker dephasing probe (`büttiker_transmission`) and watch the resonance wash out.

1.3 Compute $\sigma_{xy}(\omega)$ of the Haldane model at finite frequency and verify the tensor antisymmetry $\sigma_{xy} = -\sigma_{yx}$ at every ω .

1.4 (Harder) Build a Hofstadter magnetic supercell at flux $1/3$ with `magnetic_supercell` and verify that the lowest band's Chern number matches σ_{xy} on the magnetic cell.

1.7 Further reading

Haldane, PRL 61, 2015 (1988). Fukui, Hatsugai, Suzuki, JPSJ 74, 1674 (2005). Kuzmenko et al., PRL 100, 117401 (2008). Paulsson & Brandbyge, PRB 76, 115117 (2007). Weisse et al., Rev. Mod. Phys. 78, 275 (2006) (KPM, for the sparse module).

Module 2 — kpenvelope: Holes in Wurtzite Heterostructures

Package: kpenvelope 0.7.0 — `pip install kpenvelope` — github.com/TaN-MM-Org/kpenvelope — DOI [10.5281/zenodo.22015269](https://doi.org/10.5281/zenodo.22015269)

Methodological basis: T. M. Mahim, A. S. M. Mohsin, M. M. Rahman, “*Origin of the conflicting hole masses in the GaN/AlN two-dimensional hole gas*” (under review; code: github.com/Tanvir-Mahmud-Mahim/gan-2dhg-masses-lifetimes), on the six-band Hamiltonian of Chuang & Chang, PRB 54, 2491 (1996).

2.0 Why this package exists

Valence bands of wurtzite semiconductors are a six-band tangle of crystal-field and spin-orbit split states; a hole in a GaN quantum well has no single mass, and pretending otherwise produced a literature of conflicting numbers. kpenvelope solves the six-band k-p envelope problem on a real-space grid — symmetrized operator ordering, so the Hamiltonian is exactly Hermitian for position-dependent parameters — with hard-wall and finite-barrier (Ben Daniel-Duke) confinement, self-consistent Poisson coupling, cited parameter sets, and the v0.7 intersubband optics.

The package’s honesty rule: parameters ship only with citations. GaN and AlN sets from Rinke et al. (PRB 77, 075202 (2008)) are built in and pinned by tests; AlN’s permittivity is deliberately NaN (no vetted value shipped, none needed for a barrier), and the self-consistent solver refuses to run on it.

2.1 Conventions

Grid coordinate z in nm, energies in eV. **Hole convention: subband energies are returned descending** (the highest — the one holes occupy first — comes first). Envelopes are (n_{states} , 6 components, N grid points), normalized so $\sum_m |F_m|^2 dz = 1$.

Lesson 2.1 — Hard-wall subbands against the closed form

The shipped `demo_single_band(A)` set decouples the six bands into identical single-band problems with $m/m_0 = 1/|A|$ — so the infinite-well ladder $E_n = -(\hbar^2/2m)(n\pi/L)^2$ is an exact target, and each level must appear six-fold.

```
"""Module 2, Lesson 2.1 -- Hole subbands in a hard-wall well
(kpenvelope).
```

```
The shipped demo parameter set decouples the six-band Hamiltonian into
identical single-band problems (effective mass  $m^* = m_0/|A|$ ), so the
solver's subband energies can be checked against the infinite-well
closed form  $E_n = -(\hbar^2/2m^*)(n\pi/L)^2$  (hole convention: energies
descend). Each level appears six-fold (six decoupled components).
```

```

"""
import numpy as np

import kpenvelope
from kpenvelope import HBAR2_OVER_2M0, demo_single_band, solve_subbands

print(f"kpenvelope version: {kpenvelope.__version__}")

A = -2.0
p = demo_single_band(A=A)          #  $m^*/m_0 = 1/|A| = 0.5$ 
L = 8.0                            # nm
z = np.linspace(0.0, L, 401)
energies, envelopes = solve_subbands(p, z, n_states=18)

print(f"energies shape : {energies.shape} (descending, hole convention)")
print(f"envelopes shape: {envelopes.shape} (states, 6 components, grid)")

# closed form:  $E_n = -|A| * (\hbar^2/2m_0) * (n \pi / L)^2$ 
for n in (1, 2, 3):
    E_exact = -abs(A) * HBAR2_OVER_2M0 * (n * np.pi / L) ** 2
    E_num = energies[6 * (n - 1)]          # each level is six-fold degenerate
    print(f"n={n}: solver {E_num:+.6f} eV | closed form {E_exact:+.6f} eV "
          f"| rel. err {abs(E_num - E_exact) / abs(E_exact):.2e}")
    assert abs(E_num - E_exact) / abs(E_exact) < 2e-2 # grid-limited

# The dominant error is the effective position of the hard wall (the
# discrete boundary sits about one grid spacing outside each end), so
# the level error is FIRST order in dz: halving the spacing halves it.
# Knowing the error's origin and order is what lets you budget a grid.
z_fine = np.linspace(0.0, L, 801)
E_fine, _ = solve_subbands(p, z_fine, n_states=6)
E1_exact = -abs(A) * HBAR2_OVER_2M0 * (np.pi / L) ** 2
err_coarse = abs(energies[0] - E1_exact)
err_fine = abs(E_fine[0] - E1_exact)
print(f"ground-level error, 401 pts: {err_coarse:.2e} eV; "
      f"801 pts: {err_fine:.2e} eV; ratio {err_coarse / err_fine:.1f} "
      f"(first-order boundary error: ~2)")
assert 1.8 < err_coarse / err_fine < 2.2

# six-fold degeneracy of the ground level, exactly
spread = energies[:6].max() - energies[:6].min()
print(f"ground-multiplet spread: {spread:.2e} eV (six-fold degenerate)")
assert spread < 1e-12

# normalization contract:  $\sum_m \int |F_m|^2 dz = 1$ 
dz = z[1] - z[0]
norm = (np.abs(envelopes[0]) ** 2).sum() * dz
print(f"ground-state norm : {norm:.12f}")
assert abs(norm - 1.0) < 1e-10
print("OK: lesson 2.1 complete")

```

Verified output:

```

kpenvelope version: 0.7.0
energies shape : (18,) (descending, hole convention)
envelopes shape: (18, 6, 401) (states, 6 components, grid)
n=1: solver -0.011634 eV | closed form -0.011751 eV | rel. err 9.93e-03
n=2: solver -0.046536 eV | closed form -0.047004 eV | rel. err 9.95e-03
n=3: solver -0.104704 eV | closed form -0.105758 eV | rel. err 9.97e-03
ground-level error, 401 pts: 1.17e-04 eV; 801 pts: 5.86e-05 eV; ratio 2.0
↳ (first-order boundary error: ~2)
ground-multiplet spread: 5.98e-13 eV (six-fold degenerate)
ground-state norm      : 1.000000000000
OK: lesson 2.1 complete

```

The 1% level error is *not* the Laplacian discretization (that would be $\sim 10^{-5}$ here): it is the effective position of the discrete hard wall, about one grid spacing outside each end — which is why the refinement study shows the error halving (first order), not quartering. The lesson generalizes: before trusting any grid solver, identify the *dominant* error and its order; then you can budget a grid instead of guessing one.

Lesson 2.2 — Real GaN: bulk anchors and a quantum well

```

"""Module 2, Lesson 2.2 -- Real materials: bulk GaN anchors and an
AlN/GaN/AlN quantum well (kpenvelope).

```

```

The cited GaN parameter set (Rinke et al., Phys. Rev. B 77, 075202
(2008)) reproduces the zone-center valence splittings, and
`layered_profile` + `solve_heterostructure` build a finite-barrier
AlN/GaN/AlN well with the symmetrized Ben Daniel-Duke assembly.
"""

```

```

import numpy as np

```

```

from kpenvelope import (aln_rinke2008, gan_rinke2008, layered_profile,
                        solve_heterostructure, solve_subbands)

```

```

# --- bulk GaN zone-center splittings -----
# The 6x6 bulk block at  $k = 0$  carries the crystal-field/spin-orbit
# structure of the valence edge: A-B and A-C splittings of the Rinke
# set are 5.20 and 21.80 meV (against accepted experimental values
# near 5-6 and 22 meV; the package's test suite pins them to 0.01 meV).
from kpenvelope.hamiltonian import bulk_blocks

```

```

p = gan_rinke2008()
H0, _, _ = bulk_blocks(p, 0.0, 0.0)
E = np.sort(np.linalg.eigvalsh(H0))[:, -1] # descending; Kramers pairs
AB = (E[0] - E[2]) * 1e3
AC = (E[0] - E[4]) * 1e3
print(f"A-B splitting: {AB:.2f} meV (published Rinke value 5.20)")
print(f"A-C splitting: {AC:.2f} meV (published Rinke value 21.80)")
assert abs(AB - 5.20) < 0.01 and abs(AC - 21.80) < 0.01

```

```

# --- AlN/GaN/AlN quantum well -----
vbo = 0.85 # valence-band offset (eV), user input
zw = np.linspace(0.0, 30.0, 601)

```

```

params, edge = layered_profile(
    zw, [(10.0, aln_rinke2008(), -vbo),
         (10.0, gan_rinke2008(), 0.0),
         (10.0, aln_rinke2008(), -vbo)])
Ew, envw = solve_heterostructure(zw, params, edge, n_states=4)
print(f"top well subbands (eV): {np.array2string(Ew, precision=4)}")

# confinement check: the ground envelope lives in the GaN layer
dz = zw[1] - zw[0]
w_in = (np.abs(envw[0][:, (zw >= 10.0) & (zw <= 20.0)]) ** 2).sum() * dz
print(f"ground-state weight inside the GaN well: {w_in:.3f}")
assert w_in > 0.9
# confinement pushes the top subband below the bulk edge (holes)
assert Ew[0] < E[0]
print("OK: lesson 2.2 complete")

```

Verified output:

```

A-B splitting: 5.20 meV (published Rinke value 5.20)
A-C splitting: 21.80 meV (published Rinke value 21.80)
top well subbands (eV): [0.0138 0.0138 0.0082 0.0082]
ground-state weight inside the GaN well: 1.000
OK: lesson 2.2 complete

```

The zone-center A-B and A-C splittings come out at 5.20 and 21.80 meV — the published values of the Rinke set, which the package’s test suite pins to 0.01 meV, and which sit close to accepted experimental values (≈ 5 –6 and ≈ 22 meV). The AlN/GaN/AlN well then confines >99.9% of the ground envelope in the GaN layer. Note what the lesson does *not* do: it does not invent a valence-band offset — vbo is an explicit user input you must justify from your own literature reading.

Lesson 2.3 — Dispersion, masses, and intersubband optics

```

"""Module 2, Lesson 2.3 -- Subband dispersion, effective masses, spin
splitting, and intersubband optics (kpenvelope).

```

```

A hole mass is not one number: `local_mass` differentiates the actual
subband dispersion. Kramers pairs cannot split at  $k = 0$  (checked), and
the v0.7 intersubband tools give the dipole matrix and oscillator
strengths, pinned here to the infinite-well closed forms and the
Thomas-Reiche-Kuhn sum rule.
"""

```

```
import numpy as np
```

```

from kpenvelope import (demo_single_band, dipole_matrix, local_mass,
                        oscillator_strengths, solve_subbands,
                        spin_splitting, subband_dispersion)

```

```

A = -2.0
p = demo_single_band(A=A)
L = 8.0
z = np.linspace(0.0, L, 401)

```

```

# --- dispersion and local mass -----
kts = np.linspace(0.0, 0.6, 25)          # 1/nm, in-plane path
disp = subband_dispersion(p, z, kts, n_states=4)
k_mid, mass = local_mass(kts, disp)      # |m*|/m0 along the path
print(f"dispersion shape: {disp.shape}, local mass shape: {mass.shape}")
print(f"in-plane |mass| of the top subband at k~{k_mid[12]:.3f}/nm: "
      f"{mass[12, 0]:.4f} m0 (decoupled demo: exactly 1/|A| = 0.5)")
assert abs(mass[12, 0] - 1.0 / abs(A)) < 1e-6

# --- Kramers: no splitting at k = 0 -----
E0, _ = solve_subbands(p, z, kx=0.0, n_states=4)
print(f"spin splitting at k=0: {np.abs(spin_splitting(E0)).max():.2e} eV")
assert np.abs(spin_splitting(E0)).max() < 1e-12

# --- intersubband optics (v0.7) -----
E, env = solve_subbands(p, z, n_states=60)
d = dipole_matrix(z, env)
f = oscillator_strengths(E, d, 1.0 / abs(A)) # m*/m0 = 0.5

# the demo set is SIX-FOLD degenerate: individual elements inside a
# multiplet are basis-dependent; the multiplet-summed quantities are
# the invariants, and those are what the closed forms pin.
z12 = np.sqrt((np.abs(d[0, 6:12]) ** 2).sum())
z12_exact = 16.0 * L / (9.0 * np.pi ** 2)
f12 = f[6:12].sum()
f12_exact = 256.0 / (27.0 * np.pi ** 2)
print(f"z12 (multiplet) : {z12:.4f} nm | closed form {z12_exact:.4f} nm")
print(f"f12 (multiplet) : {f12:.4f} | closed form {f12_exact:.4f}")
print(f"TRK f-sum : {f.sum():.4f} (exact: 1)")
z13 = np.sqrt((np.abs(d[0, 12:18]) ** 2).sum())
print(f"parity-forbidden z13: {z13:.2e} nm (~0)")
assert abs(z12 - z12_exact) / z12_exact < 0.01
assert abs(f12 - f12_exact) < 1e-3
assert 0.995 < f.sum() < 1.0005
assert z13 < 1e-9
print("OK: lesson 2.3 complete")

```

Verified output:

```

dispersion shape: (25, 4), local mass shape: (24, 4)
in-plane |mass| of the top subband at k~0.312/nm: 0.5000 m0 (decoupled demo:
↳ exactly 1/|A| = 0.5)
spin splitting at k=0: 3.01e-13 eV
z12 (multiplet) : 1.4482 nm | closed form 1.4410 nm
f12 (multiplet) : 0.9607 | closed form 0.9607
TRK f-sum : 0.9992 (exact: 1)
parity-forbidden z13: 3.12e-12 nm (~0)
OK: lesson 2.3 complete

```

Three habits to take away. First, a hole mass is a *function*, not a number — `local_mass` differentiates the actual dispersion, and on the decoupled demo it returns exactly $1/|A|$ at every k , which is your check that the machinery is sound before you point it at a coupled set. Second, Kramers pairs cannot split at $k = 0$, and the solver's 10^{-13} is that theorem, numerically. Third, the intersubband dipoles: the demo set is six-fold degenerate, so individual matrix elements inside a multiplet are basis-dependent noise — only the multiplet-summed quantities are invari-

ant, and those land on the infinite-well closed forms $z_{12} = 16L/9\pi^2$ (0.5% at this grid) and $f_{12} = 256/27\pi^2$ (four figures), with the Thomas-Reiche-Kuhn sum at 0.9992. When your own basis has degeneracies, sum over them before interpreting anything.

2.4 Pitfalls

Energies descend (hole convention) — sorting ascending silently reverses your subband indices. The oscillator-strength `mass_ratio` is the *confinement-direction* mass of the band; for coupled six-band states no single mass exists, and the TRK sum is then not 1 (the strengths remain well-defined matrix elements). Strain requires cited deformation potentials — none ship, on purpose.

2.5 Exercises

2.1 Verify the origin-shift gauge identity yourself: shift z by any a and confirm every off-diagonal dipole element is unchanged while the diagonal shifts by exactly a — on the *coupled* `gan_rinke2008()` well.

2.2 Compute `splitting_vs_k` for a symmetric and an asymmetric (add a linear potential) well and confirm Rashba-type splitting appears only in the asymmetric case.

2.3 Converge the AlN/GaN/AlN ground subband against grid spacing and well width; identify when the hard-wall solver (`solve_subbands`) becomes an acceptable substitute for the finite-barrier one.

2.6 Further reading

Chuang & Chang, PRB 54, 2491 (1996). Rinke et al., PRB 77, 075202 (2008). Vurgaftman & Meyer, J. Appl. Phys. 94, 3675 (2003) (for building your own cited parameter sets).

Part II — Measurement and Inference

Module 3 — ramansep: Strain or Doping? Ask Two Phonons

Package: ramansep 0.7.0 — pip install ramansep — github.com/TaN-MM-Org/ramansep — DOI [10.5281/zenodo.22014913](https://doi.org/10.5281/zenodo.22014913)

Methodological basis: T. M. Mahim, M. M. Rahman, “Two Raman phonons quantify the fixed edge charge left by patterning monolayer transition metal dichalcogenides” (under review; companion data DOI [10.5281/zenodo.21778170](https://doi.org/10.5281/zenodo.21778170)), building on the vector-decomposition idea of Lee et al., Nat. Commun. 3, 1024 (2012).

3.0 Why this package exists

A Raman peak shifts when a 2D flake is strained — and also when it is doped. One mode cannot tell the two causes apart; two modes with *different* lever arms can, by a 2×2 linear inversion. ramansep implements that inversion with the parts that usually get skipped: full uncertainty propagation, conditioning diagnostics, a peak-fitting front end, an overdetermined multimode generalization with a per-pixel model check, a Bayesian whole-map inversion with spatial priors, and — v0.7 — the calibration tool that fits *your own* lever arms from reference measurements.

The coefficient policy is the honesty rule again: the package ships only cited example sets (two for monolayer MoS₂, one for graphene), each with its provenance and its caveats in the docstring, and refuses to pretend lever arms are universal. Your material, excitation and substrate need your calibration — and since v0.7 the package hands you the tool for it.

3.1 Conventions

Shifts in cm^{-1} relative to a pristine reference (softening = negative). Strain and density come out in whatever units the coefficient matrix implies; for the built-in MoS₂ sets that is percent biaxial strain and electron density in 10^{13} cm^{-2} . The graphene set deliberately returns a doping *coordinate* in cm^{-1} rather than a fake universal density — read its docstring before using it.

Lesson 3.1 — The inversion, exact round trip, and the paper’s edge charge

```
"""Module 3, Lesson 3.1 -- The two-mode inversion: strain and carrier
density from Raman peak shifts (ramansep).
```

```
Two Raman modes with different strain lever arms and doping responses
form a linear, invertible probe of (strain, density). We use the cited
MoS2 A'1 + 2LA(M) coefficient set, verify the exact forward/inverse
round trip, and propagate measurement uncertainties.
```

```
"""
```

```
import numpy as np
```

```

import ramansep as rs

print(f"ramansep version: {rs.__version__}")

coeff = rs.mos2_a1_2la()          # cited set (provenance in .reference)
model = rs.SeparationModel(coeff)
print(f"modes: {coeff.model_name} + {coeff.mode2_name}")
print(f"lever-arm matrix K:\n{np.array(coeff.matrix())}")
print(f"condition number: {model.condition_number:.2f}")

# --- forward, then invert: the round trip is exact -----
rng = np.random.default_rng(0)
strain = rng.uniform(-0.2, 0.2, (4, 4))    # percent biaxial
density = rng.uniform(0.0, 1.0, (4, 4))    # 1e13 cm^-2 electrons
dw1, dw2 = model.forward(strain, density)
res = model.invert(dw1, dw2)
print(f"round-trip |strain error| max : "
      f"{np.abs(res.strain - strain).max():.2e} %")
print(f"round-trip |density error| max: "
      f"{np.abs(res.density - density).max():.2e} x1e13 cm^-2")
assert np.allclose(res.strain, strain, atol=1e-12)
assert np.allclose(res.density, density, atol=1e-12)

# --- uncertainty propagation -----
# 1-sigma peak-shift uncertainties of 0.15 cm^-1 on both modes
res_u = model.invert(dw1, dw2, sigma1=0.15, sigma2=0.15)
print(f"strain sigma (uniform 0.15 cm^-1 shifts): "
      f"{res_u.strain_sigma.flat[0]:.4f} %")
print(f"density sigma : "
      f"{res_u.density_sigma.flat[0]:.4f} x1e13 cm^-2")
print(f"strain-density correlation : "
      f"{res_u.correlation.flat[0]:+.3f}")
print("(correlations near +/-1 would mean the mode pair barely "
      "separates the two causes)")

# --- reproduce the source paper's edge-charge separation -----
# A 0.5 cm^-1 A'1 redshift with the 2LA(M) unmoved is the paper's edge
# signature: pure charge, no strain.
res_edge = model.invert(np.array([-0.5]), np.array([0.0]))
print(f"A'1 -0.5 cm^-1, 2LA(M) 0 -> strain {res_edge.strain[0]:+.4f} %, "
      f"density {res_edge.density[0]:.3f} x1e13 cm^-2 "
      f"(= {res_edge.density[0] * 1e13:.2g} cm^-2)")
print("OK: lesson 3.1 complete")

```

Verified output:

```

ramansep version: 0.7.0
modes: A'1 + 2LA(M)
lever-arm matrix K:
[[ -5.1  -2.2]
 [ -20.9   0. ]]
condition number: 10.07
round-trip |strain error| max : 2.78e-17 %
round-trip |density error| max: 2.22e-16 x1e13 cm^-2

```

```

strain sigma (uniform 0.15 cm-1 shifts): 0.0072 %
density sigma : 0.0702 x1e13 cm-2
strain-density correlation : -0.237
(correlations near +/-1 would mean the mode pair barely separates the two causes)
A'1 -0.5 cm-1, 2LA(M) 0 -> strain +0.0000 %, density 0.227 x1e13 cm-2 (= 2.3e+12
↳ cm-2)
OK: lesson 3.1 complete

```

The forward→invert round trip is exact to machine precision — the inversion is a matrix inverse, and the test proves the plumbing. With 0.15 cm⁻¹ shift noise on both modes this mode pair delivers 0.007% strain and 7×10¹¹ cm⁻² density resolution, with a mild −0.24 correlation: the A'1+2LA(M) pair genuinely separates the two causes (that is why the source paper chose it — the 2LA(M) strain arm is 4× larger while its doping response is negligible). The last block reproduces the paper's headline separation: a 0.5 cm⁻¹ A'1 redshift with 2LA(M) unmoved is *pure charge*, 2.3×10¹² cm⁻², zero strain.

Lesson 3.2 — From raw spectra to values

```

"""Module 3, Lesson 3.2 -- From raw spectra to strain/density values
(ramansep).

The fitting front end closes the gap between a measured spectrum and
the inversion: synthesize a noisy two-peak Raman spectrum with a KNOWN
(strain, density) state, fit both modes with `fit_two_modes`, and feed
the shifts straight into the inversion. The recovered state must agree
with the truth within the propagated uncertainty.
"""

import numpy as np

import ramansep as rs
from ramansep import lorentzian

rng = np.random.default_rng(1)
coeff = rs.mos2_a1_2la()
model = rs.SeparationModel(coeff)

# pristine reference frequencies of the two modes (cm-1). For real
# data these come from YOUR pristine flake; here they are stand-ins.
REF1, REF2 = 404.0, 450.0

# ground truth at one map pixel
strain_true, density_true = 0.10, 0.60 # percent, 1e13 cm-2
dw1, dw2 = model.forward(strain_true, density_true)
print(f"true shifts: dw1 = {float(dw1):+.3f}, dw2 = {float(dw2):+.3f} cm-1")

# --- synthesize the measured spectrum -----
x = np.arange(380.0, 470.0, 0.35) # spectrometer axis
y = (lorentzian(x, REF1 + float(dw1), 5.0, 900.0)
     + lorentzian(x, REF2 + float(dw2), 9.0, 420.0)
     + 60.0 # baseline
     + rng.normal(0.0, 6.0, x.size)) # shot/read noise

```

```

# --- fit both modes and invert -----
dw1_fit, dw2_fit, s1, s2, fit1, fit2 = rs.fit_two_modes(
    x, y, window1=(390.0, 425.0), window2=(432.0, 468.0),
    ref1=REF1, ref2=REF2)
print(f"fitted shifts: dw1 = {dw1_fit:+.3f} +/- {s1:.3f}, "
      f"dw2 = {dw2_fit:+.3f} +/- {s2:.3f} cm^-1")
print(f"fit converged: {fit1.converged} / {fit2.converged}; "
      f"residual rms {fit1.residual_rms:.1f} / {fit2.residual_rms:.1f} counts")

res = model.invert(np.array([dw1_fit]), np.array([dw2_fit]),
                  sigma1=np.array([s1]), sigma2=np.array([s2]))
print(f"recovered strain : {res.strain[0]:+.4f} +/- "
      f"{res.strain_sigma[0]:.4f} % (true {strain_true:+.3f})")
print(f"recovered density: {res.density[0]:+.4f} +/- "
      f"{res.density_sigma[0]:.4f} x1e13 cm^-2 (true {density_true:.3f})")

# agreement within 3 sigma of the propagated uncertainty
assert abs(res.strain[0] - strain_true) < 3 * res.strain_sigma[0]
assert abs(res.density[0] - density_true) < 3 * res.density_sigma[0]
print("OK: lesson 3.2 complete")

```

Verified output:

```

true shifts: dw1 = -1.830, dw2 = -2.090 cm^-1
fitted shifts: dw1 = -1.824 +/- 0.007, dw2 = -2.109 +/- 0.021 cm^-1
fit converged: True / True; residual rms 6.1 / 5.9 counts
recovered strain : +0.1009 +/- 0.0010 % (true +0.100)
recovered density: +0.5949 +/- 0.0040 x1e13 cm^-2 (true 0.600)
OK: lesson 3.2 complete

```

The Levenberg-Marquardt front end (analytic Jacobians, multi-start) recovers the peak shifts to millikayser precision from a realistically noisy spectrum, and the inversion returns strain and density within the propagated 1σ of the ground truth. Note the windows must not overlap — a shared shoulder would be counted twice, and the fitter refuses.

Lesson 3.3 — More modes, model checking, and your own calibration

```

"""Module 3, Lesson 3.3 -- Beyond two modes, and calibrating your own
lever arms (ramansep).

```

```

Three or more modes make the inversion overdetermined: the redundancy
buys smaller uncertainties AND a per-pixel chi-square model check that
no exactly-determined inversion can offer. And when no literature
coefficient set fits your material, `calibrate_lever_arms` (v0.7)
fits the matrix from reference measurements you control.
"""

```

```

import numpy as np

import ramansep as rs

rng = np.random.default_rng(2)

```

```

# --- an overdetermined three-mode inversion -----
# lever arms of three hypothetical modes (units as usual)
K = np.array([[ -5.1, -2.2],
               [-20.9, 0.0],
               [-4.3, -0.33]])
mm = rs.MultiModeModel(K, mode_names=["A'1", "2LA(M)", "E'"])
strain, density = 0.15, 0.5
shifts_clean = K @ np.array([strain, density])
sig = np.array([0.10, 0.15, 0.10])
shifts = shifts_clean + rng.normal(0.0, sig)
res = mm.invert(shifts, sigmas=sig)
print(f"3-mode GLS: strain {float(res.strain):+.4f} +/- "
      f"{float(res.strain_sigma):.4f} %, density "
      f"{float(res.density):+.4f} +/- {float(res.density_sigma):.4f}")
print(f"model check: chi2 = {float(res.chi2_map):.3f} on {res.dof} dof, "
      f"p = {float(res.p_value):.3f} (the two-cause model holds)")

# a pixel that VIOLATES the two-cause model (a third latent variable)
bad = shifts + np.array([0.0, 0.0, 1.2])
res_bad = mm.invert(bad, sigmas=sig)
print(f"violated pixel: p = {float(res_bad.p_value):.2e} (flagged)")
assert float(res_bad.p_value) < 1e-3 < float(res.p_value)

# which mode pair is best? a computation, not a habit:
ranking = rs.compare_mode_sets(K, sigmas=sig,
                               mode_names=["A'1", "2LA(M)", "E'"])
best = ranking[0]
print(f"best 2-mode subset by D-optimality: {best['names']}")

# --- calibrate your own lever arms (v0.7) -----
# reference states you control: a strain stage sweep (density 0)
# plus gated points (strain 0) -- together they span the plane.
eps_ref = np.array([0.00, 0.05, 0.10, 0.20, 0.00, 0.00, 0.00])
rho_ref = np.array([0.00, 0.00, 0.00, 0.00, 0.30, 0.60, 1.00])
K_true = np.array([[ -5.1, -2.2], [-20.9, 0.0]])
meas = (np.column_stack([eps_ref, rho_ref]) @ K_true.T
        + rng.normal(0.0, 0.08, (7, 2)))
cal = rs.calibrate_lever_arms(eps_ref, rho_ref, meas, sigmas=0.08,
                             mode_names=["A'1", "2LA(M)"])
print("calibrated lever arms (true in brackets):")
for i in range(2):
    print(f" mode {i + 1}: strain {cal.K[i, 0]:+.2f} +/- "
          f"{cal.K_sigma[i, 0]:.2f} [{K_true[i, 0]:+.1f}], doping "
          f"{cal.K[i, 1]:+.2f} +/- {cal.K_sigma[i, 1]:.2f} "
          f"[{K_true[i, 1]:+.1f}]")
print(f"chi2/dof: {(cal.chi2 / cal.chi2_dof).round(2)} (near 1: model "
      f"and stated sigmas consistent)")
# each fitted arm within 3 sigma of the generating value
assert np.all(np.abs(cal.K - K_true) < 3.0 * cal.K_sigma + 1e-12)

# a strain-only sweep is refused: doping arms are not identifiable
try:
    rs.calibrate_lever_arms(eps_ref[:4], rho_ref[:4], meas[:4],

```

```

                                sigmas=0.08)
except ValueError as e:
    print(f"strain-only sweep refused: {str(e)[:60]}...")
print("OK: lesson 3.3 complete")

```

Verified output:

```

3-mode GLS: strain +0.1543 +/- 0.0070 %, density +0.4834 +/- 0.0483
model check: chi2 = 0.088 on 1 dof, p = 0.767 (the two-cause model holds)
violated pixel: p = 3.47e-29 (flagged)
best 2-mode subset by D-optimality: ["A'1", '2LA(M)']
calibrated lever arms (true in brackets):
  mode 1: strain -5.06 +/- 0.35 [-5.1], doping -2.24 +/- 0.07 [-2.2]
  mode 2: strain -20.58 +/- 0.35 [-20.9], doping +0.04 +/- 0.07 [+0.0]
chi2/dof: [1.72 0.76] (near 1: model and stated sigmas consistent)
strain-only sweep refused: reference states are collinear in the (strain, density)
↳ plan...
OK: lesson 3.3 complete

```

Three modes buy two things two modes cannot: the Gauss–Markov GLS estimate has smaller errors, and the redundancy yields a per-pixel chi-square with $m-2$ degrees of freedom — the $p = 3 \times 10^{-29}$ pixel is one where strain and density *alone cannot explain the data* (a third latent variable, a phase boundary, a bad fit), something an exactly-determined inversion can never detect. `compare_mode_sets` then turns mode choice into a computation. Finally the calibration: seven reference points (a strain sweep plus gated points) recover both modes’ lever arms within their reported uncertainties — and the strain-only subset is *refused*, because collinear reference states cannot identify the doping arms. That refusal is the package protecting you from a silently meaningless fit.

3.4 Pitfalls

Check `condition_number` before trusting an inversion — and note it is unit-dependent (the graphene set’s ~ 92 is an artifact of mixed units; the docstring tells you to raise `condition_warn` and what the physically meaningful noise amplification is). The 2LA(M) doping coefficient is *unmeasured*, set to zero with a documented 16%-of-edge-charge consequence bound — carry that systematic in your error budget. Without measurement sigmas, `calibrate_lever_arms` needs $n \geq 3$ reference points; with $n = 2$ it would have to invent an error bar, so it refuses.

3.5 Exercises

3.1 Propagate asymmetric per-mode uncertainties ($\sigma_1 \neq \sigma_2$) and find the shift-noise ratio at which the $E'+A'1$ pair becomes competitive with $A'1+2LA(M)$ for *density* (not strain).

3.2 Run `bayesian_map_inversion` on a synthetic 32×32 map with a sharp strain step; study how the smoothness weight λ trades noise suppression against edge blurring, and verify $\lambda=0$ reproduces the per-pixel GLS maps.

3.3 Design a minimal calibration: what is the smallest set of (strain, density) reference states that keeps both arms’ uncertainties below 10% for your assumed shift noise? Use the exact WLS covariance rather than simulation.

3.6 Further reading

Lee, Ahn, Shim, Lee, Ryu, Nat. Commun. 3, 1024 (2012). Chakraborty et al., PRB 85, 161403(R) (2012). Michail et al., ACS Appl. Mater. Interfaces 16, 49602 (2024). Rue & Held, *Gaussian Markov Random Fields* (2005) (for the Bayesian map prior).

Module 4 — sparq-triage: Is It a Single Photon Source?

Package: sparq-triage 0.5.0 — pip install sparq-triage — github.com/TaN-MM-Org/sparq-triage — DOI [10.5281/zenodo.22278041](https://doi.org/10.5281/zenodo.22278041)

Methodological basis: the manuscript “*Closed-loop, event-driven machine learning for autonomous triage of single-photon emitters*” (companion code: github.com/Tanvir-Mahmud-Mahim/a-spiking-RL-triage-of-solid-state-single-photon-emitters). The NumPy/SciPy physics core taught here needs no PyTorch; the spiking-network estimators and reinforcement-learning triage loop live behind the [ml] extra.

4.0 Why this package exists

The workhorse question of quantum-emitter labs — is $g_2(0) < 1/2$? — hides real statistical subtlety: Poisson counts, a dip convolved with the instrument response, a bunching shoulder, background, and the economics of acquisition time. sparq provides the physics twin (exact Poisson statistics of the HBT histogram over cited platform priors for NV, hBN, GaN, SiV), a numerically exact three-level master-equation reference, and a set of inference tools that answer the question three honest ways — plus Wald’s sequential test to answer it *fast*.

4.1 Conventions

Delays in ns; the standard analysis grid is 121 bins of 1 ns spanning ± 60.5 ns (odd count, so one bin is centered at zero). Site photophysics is (τ_1, τ_2, a) : antibunching time, bunching time, bunching amplitude; ρ is the signal fraction (background enters as $g_2 \rightarrow 1 - \rho^2(1 - g_2)$).

Lesson 4.1 — The twin and the exact reference

```
"""Module 4, Lesson 4.1 -- The HBT twin and its exact master-equation
reference (sparq-triage).
```

```
The analytic two-exponential g2 law (antibunching dip + metastable
bunching shoulder) is cross-checked against the numerically exact
three-level master equation, and the histogram twin's expected counts
follow the exact Poisson statistics of an HBT acquisition.
```

```
"""
```

```
import numpy as np
```

```
import sparq
```

```
print(f"sparq-triage version: {sparq.__version__}")
```

```
# --- analytic g2 vs the exact master equation -----
# The nominal (tau1, tau2, a) map to CTMC rates; the EXACT
```

```

# two-exponential parameters of that CTMC come back slightly
# renormalized from the eigen-decomposition (`effective_params`).
# With those, the analytic law equals the exact master equation to
# machine precision -- the package's core cross-validation.
tau1, tau2, a = 15.0, 250.0, 0.4 # ns, ns, bunching amplitude
rates = sparq.rates_from_site(tau1, tau2, a)
tle, t2e, ae = sparq.effective_params(*rates)
print(f"nominal (tau1, tau2, a): ({tau1:.2f}, {tau2:.1f}, {a:.3f})")
print(f"effective (tau1, tau2, a): ({tle:.2f}, {t2e:.1f}, {ae:.3f})")
tau = np.array([0.0, 2.0, 10.0, 40.0, 150.0, 600.0])
g2_analytic = sparq.g2_three_level(tau, tle, t2e, ae)
g2_master = sparq.g2_exact(tau, *rates)
print(" tau (ns) | analytic g2 | master-equation g2")
for t, ga, gm in zip(tau, g2_analytic, g2_master):
    print(f" {t:.6f} | {ga:.6f} | {gm:.6f}")
assert np.abs(g2_analytic - g2_master).max() < 1e-10
print("two independent code paths agree to machine precision")

# --- single- vs multi-emitter zero-delay values -----
for n in (1, 2, 3):
    g0 = sparq.g2_zero(tau1, tau2, a, n_emitters=n, rho=1.0)
    print(f"n_emitters = {n}: g2(0) = {g0:.4f} "
          f"(the n-emitter floor 1 - 1/n = {1 - 1 / n:.4f})")

# --- the histogram twin -----
cfg = sparq.HBTConfig() # 121 bins, +/-60.5 ns, 1 ns bins
site = sparq.EmitterSite(dict(tau1=tau1, tau2=tau2, a=a, rho=0.95,
                              rate_kcps=150.0, blinking=False),
                          n_emitters=1)
print(f"site g2(0) (with rho=0.95 background): {site.g2_0:.4f}")
T_s = 30.0 # seconds of acquisition
mu = sparq.expected_histogram(site, T_s, cfg)
rng = np.random.default_rng(0)
h = rng.poisson(mu)
mid = cfg.n_bins // 2
far = np.abs(cfg.bin_centers) >= 0.65 * cfg.tau_max
print(f"expected counts: center bin {mu[mid]:.1f}, far-delay mean "
      f"{mu[far].mean():.1f}")
print(f"sampld counts: center bin {h[mid]}, far-delay mean "
      f"{h[far].mean():.1f}")
print(f"raw dip ratio (sampld): {h[mid] / h[far].mean():.3f}")
assert mu[mid] < 0.6 * mu[far].mean() # a real dip
print("OK: lesson 4.1 complete")

```

Verified output:

```

sparq-triage version: 0.5.0
nominal (tau1, tau2, a): (15.00, 250.0, 0.400)
effective (tau1, tau2, a): (14.49, 177.3, 0.447)
tau (ns) | analytic g2 | master-equation g2
0.0 | -0.000000 | -0.000000
2.0 | 0.181523 | 0.181523
10.0 | 0.696710 | 0.696710
40.0 | 1.264924 | 1.264924

```

```

150.0 | 1.191584 | 1.191584
600.0 | 1.015141 | 1.015141
two independent code paths agree to machine precision
n_emitters = 1: g2(0) = 0.0000 (the n-emitter floor 1 - 1/n = 0.0000)
n_emitters = 2: g2(0) = 0.5000 (the n-emitter floor 1 - 1/n = 0.5000)
n_emitters = 3: g2(0) = 0.6667 (the n-emitter floor 1 - 1/n = 0.6667)
site g2(0) (with rho=0.95 background): 0.0975
expected counts: center bin 21.9, far-delay mean 210.4
sampled counts: center bin 21, far-delay mean 213.1
raw dip ratio (sampled): 0.099
OK: lesson 4.1 complete

```

The most instructive line is the parameter renormalization: nominal (15, 250, 0.4) becomes effective (14.49, 177.3, 0.447). The nominal values parameterize the CTMC *rates*; the exact two-exponential parameters of that CTMC come back from the eigen-decomposition slightly shifted. With the effective parameters, the analytic law equals the exact master equation to machine precision — two independent code paths, one physics. If you fit (τ_1 , τ_2 , a) to data, you are fitting the effective parameters; know which numbers you are quoting. The n-emitter floor $1-1/n$ and the twin's Poisson dip complete the picture.

Lesson 4.2 — Three honest answers to “what is $g_2(0)$?”

```

"""Module 4, Lesson 4.2 -- Three honest answers to "what is g2(0)?"
(sparq-triage).

The same twin-generated histogram is analyzed three ways: the
bootstrap pipeline (shot noise through every analysis step), the Wilks
profile likelihood (model-based, exact Poisson), and the v0.5 exact
Bayesian posterior of the raw central-window ratio (closed-form
beta-prime, no sampling). Each answers a slightly different question;
all three should bracket the truth here.
"""

import numpy as np

import sparq

cfg = sparq.HBTConfig()
site = sparq.EmitterSite(dict(tau1=15.0, tau2=250.0, a=0.4, rho=0.95,
                             rate_kcps=150.0, blinking=False),
                          n_emitters=1)
print(f"true model g2(0) (background-limited): {site.g2_0:.4f}")

T_s = 60.0
rng = np.random.default_rng(3)

# Raw correlator data arrive on the instrument's own grid -- wider
# than the analysis window and offset (nobody centers hardware delays
# at exactly zero). analyze_histogram locates the dip and re-bins.
cfg_raw = sparq.HBTConfig(tau_max=90.0, n_bins=360, sigma_irf=0.35)
mu_raw = sparq.expected_histogram(site, T_s, cfg_raw)
delay_raw = cfg_raw.bin_centers + 7.0 # 7 ns hardware offset
counts_raw = rng.poisson(mu_raw).astype(float)

```

```

# --- route 1: the bootstrap pipeline -----
res = sparq.analyze_histogram(delay_raw, counts_raw, T_s=T_s,
                             n_bootstrap=100, seed=0)
print(f"[bootstrap] dip located at {res['center']:+.2f} ns "
      f"(true offset +7.00)")
print(f"[bootstrap] g2(0) = {res['g2_0']:.3f} "
      f"68% CI ({res['g2_0_low']:.3f}, {res['g2_0_high']:.3f}) "
      f"single-emitter confident: {res['single_emitter_confident']}")

# --- route 2: Wilks profile likelihood -----
# re-bin the raw data onto the standard analysis grid, dip-centered
from sparq.datasets import rebin_real, robust_flat_rate
h, center = rebin_real(delay_raw, counts_raw, cfg)
h = h.astype(float)
r_hat = robust_flat_rate(h, cfg, T_s)
prof = sparq.profile_likelihood_ci(h, T_s, r_hat, cfg, level=0.95)
print(f"[profile] g2(0) = {prof['g2_hat']:.3f} "
      f"95% CI ({prof['lo']:.3f}, {prof['hi']:.3f})")

# --- route 3: exact Bayesian posterior (v0.5) -----
post = sparq.bayesian_g2(h, cfg) # Jeffreys prior
lo, hi = post.credible_interval(0.95)
print(f"[Bayes] raw window g2 median = {post.median():.3f} "
      f"95% credible ({lo:.3f}, {hi:.3f})")
print(f"[Bayes] P(g2 < 0.5 | data) = {post.prob_below(0.5):.6f}")

# NOTE the estimand difference, stated honestly: routes 1-2 estimate
# the model's g2(0); route 3 estimates the RAW 1-ns window average,
# which sits on top of the dip and upper-bounds g2(0).

# --- background correction -----
# the twin included rho = 0.95; correct the raw Bayes median for it
corr = sparq.background_corrected_g2(post.median(), rho=0.95)
print(f"background-corrected raw g2: {corr['g2_corrected']:.3f}")

assert res['g2_0'] < 0.5 and prof['hi'] < 0.5
assert post.prob_below(0.5) > 0.99
print("OK: lesson 4.2 complete")

```

Verified output:

```

true model g2(0) (background-limited): 0.0975
[bootstrap] dip located at +6.75 ns (true offset +7.00)
[bootstrap] g2(0) = 0.074 68% CI (0.062, 0.080) single-emitter confident: True
[profile] g2(0) = 0.075 95% CI (0.070, 0.086)
[Bayes] raw window g2 median = 0.094 95% credible (0.067, 0.126)
[Bayes] P(g2 < 0.5 | data) = 1.000000
background-corrected raw g2: 0.000
OK: lesson 4.2 complete

```

Three routes, three estimands — and the lesson is precisely their differences. The bootstrap ($g2 = 0.074$) and profile likelihood (0.075, with a Wilks 95% CI) estimate the *model's* zero-delay value; the exact Bayesian posterior (median 0.094) estimates the *raw 1-ns window average*, which sits on top of the dip and upper-bounds $g2(0)$ — the docstring says so, and now you

know why the third number is slightly higher, not “wrong”. $P(g_2 < \frac{1}{2} \mid \text{data}) = 1.000000$ is the closed-form verdict probability. The pipeline also located the deliberate +7 ns hardware offset on its own. One honest subtlety: the true model value is 0.0975, and the fit-based routes land a shade below it — dip-shape fitting at finite counts with an IRF has its own small biases; the profile CI’s job is to be *calibrated*, and its 95% interval here brackets most of that.

Lesson 4.3 — Stop measuring when you know

"""Module 4, Lesson 4.3 -- Sequential certification: stop acquiring the moment the evidence suffices (sparq-triage)."""

Wald's SPRT on accumulating HBT histograms: after every data increment the exact Poisson log-likelihood ratio between the single-emitter and pair hypotheses is updated, and acquisition stops at the error-rate thresholds. Bright single emitters certify in a fraction of the fixed dwell a conventional protocol would spend.

```
import numpy as np

import sparq

cfg = sparq.HBTConfig()
params = dict(tau1=15.0, tau2=250.0, a=0.3, rho=0.95,
              rate_kcps=250.0, blinking=False)
site1 = sparq.EmitterSite(params, 1)      # accept hypothesis
site2 = sparq.EmitterSite(params, 2)      # reject hypothesis
print(f'hypotheses: g2(0) = {site1.g2_0:.3f} (single) vs '
      f'{site2.g2_0:.3f} (pair)')

rng = np.random.default_rng(4)
dt = 0.05                                # seconds per increment

def run_sprt(true_site, seed):
    rng = np.random.default_rng(seed)
    cert = sparq.SPRTCertifier(site1, site2, cfg=cfg,
                               alpha=0.05, beta=0.05)

    t = 0.0
    while True:
        counts = rng.poisson(sparq.expected_histogram(true_site, dt, cfg))
        verdict = cert.update(counts, dt)
        t += dt
        if verdict != "continue" or t > 30.0:
            return verdict, t

# --- certify a true single emitter -----
times1 = []
for s in range(20):
    v, t = run_sprt(site1, seed=100 + s)
    assert v in ("accept", "reject")
    times1.append((v, t))
acc = sum(1 for v, _ in times1 if v == "accept")
```

```

mean_t = np.mean([t for _, t in times1])
print(f"true single emitter, 20 runs: {acc}/20 accepted, "
      f"mean decision time {mean_t:.2f} s")

# --- and a true pair -----
times2 = []
for s in range(20):
    v, t = run_sprt(site2, seed=300 + s)
    times2.append((v, t))
rej = sum(1 for v, _ in times2 if v == "reject")
mean_t2 = np.mean([t for _, t in times2])
print(f"true pair,          20 runs: {rej}/20 rejected, "
      f"mean decision time {mean_t2:.2f} s")

print(f"(a fixed-dwell protocol would spend ~30 s per site; the SPRT "
      f"decided in well under a second here)")
assert acc >= 18 and rej >= 18 # ~nominal 5% error rates
assert mean_t < 5.0 and mean_t2 < 5.0
print("OK: lesson 4.3 complete")

```

Verified output:

```

hypotheses: g2(0) = 0.098 (single) vs 0.549 (pair)
true single emitter, 20 runs: 19/20 accepted, mean decision time 0.11 s
true pair,          20 runs: 18/20 rejected, mean decision time 0.11 s
(a fixed-dwell protocol would spend ~30 s per site; the SPRT decided in well under a
↳ second here)
OK: lesson 4.3 complete

```

The SPRT decided in ~0.11 s what a fixed protocol would dwell 30 s on — a 250× economy at the stated 5% error rates (19/20 and 18/20 across these seeded runs, consistent with the nominal rates). This is the statistical engine behind the paper’s autonomous triage: spend photons where the decision is hard, not where it is easy.

4.4 Pitfalls

`analyze_histogram` expects the *instrument’s* raw grid (wider than ± 60.5 ns, arbitrary offset) — feeding it data already on the analysis grid loses the edge bins at re-binning (this course’s own first draft made that mistake; the lesson now shows the right call). The SPRT’s guarantees are for fully specified hypotheses; with plug-in nuisance parameters they are approximate, and the package documents the empirical check. Background correction inverts $g2_meas = 1 + \rho^2(g2-1)$ and truncates at the physical floor *without hiding the untruncated value* — quote both.

4.5 Exercises

4.1 Sweep acquisition time and plot how the three routes’ intervals shrink; identify where the Bayesian window estimand’s dip-averaging bias becomes visible against the model-based routes.

4.2 Give the SPRT a *wrong* pair hypothesis (τ_1 off by $2\times$) and measure the realized error rates — quantify the plug-in caveat yourself.

4.3 Use `bayesian_g2`'s `n_center_bins=3` and watch the posterior shift upward as the window climbs the dip walls; reconcile the shift with the known dip shape.

4.6 Further reading

Wald & Wolfowitz, *Ann. Math. Stat.* 19, 326 (1948). Brouri et al., *Opt. Lett.* 25, 1294 (2000). Simon's PPT criterion is not needed here — that is Module 7's world; this module's Bayesian machinery is conjugate Gamma/beta-prime, see e.g. Gelman et al., *Bayesian Data Analysis*, ch. 2.

Module 5 — absnoise: The Noise Floor of Andreev Thermometry

Package: absnoise 0.6.0 — `pip install absnoise` — github.com/TaN-MM-Org/absnoise — DOI [10.5281/zenodo.22048608](https://doi.org/10.5281/zenodo.22048608)

Methodological basis: T. M. Mahim, A. S. M. Mohsin, M. M. Rahman, “*Andreev occupation noise sets the sensitivity limit of proximity Josephson thermal detectors*” (manuscript; code: github.com/Tanvir-Mahmud-Mahim/andreev-occupation-noise). Junction recipes from Jung et al., Phys. Rev. Applied 26, 014078 (2026) (arXiv:2503.06850); graphene thermal parameters per Lee et al., Nature 586, 42 (2020).

5.0 Why this package exists

A proximity Josephson junction is a thermometer: its Andreev bound states’ occupations, and through them the supercurrent, depend on temperature. But those same occupations *fluctuate*, and that telegraph noise is the intrinsic sensitivity floor of the whole device class. `absnoise` computes the floor and everything around it: the exact finite-length Andreev spectrum, the closed-form short-junction ensemble, the BCS gap solved from the gap equation itself (the popular tanh interpolation misrepresents $d\Delta/dT$ at low T by an order of magnitude — so the package refuses to use it), master-equation and Monte-Carlo noise, and device budgets up to single-photon calorimetry. Since v0.6 it also fits *your* junction’s (I_{c0} , T_c^* , τ) from a measured $I_c(T)$ curve.

Lesson 5.1 — Recipes, junctions, and the thermometry bound

```
"""Module 5, Lesson 5.1 -- Junction recipes and the short-junction
ensemble (absnoise).
```

```
The six cited graphene-junction contact recipes (Jung et al.,
arXiv:2503.06850, Table I) drive the closed-form short-junction
ensemble: Andreev levels, critical current, and the intrinsic
occupation-noise limit of Andreev thermometry -- the Cauchy-Schwarz
bound  $\text{var}(T) \geq 2 \text{ kB } T^2 \tau_A / (C_A t)$ , which uniform-transparency
short junctions saturate exactly.
```

```
"""
```

```
import numpy as np
```

```
import absnoise as ab
```

```
print(f"absnoise version: {ab.__version__}")
```

```
print(f"{len(ab.RECIPES)} cited junction recipes:")
for r in ab.RECIPES:
```

```

    print(f"   {r.label:12s} Tc* = {r.Tc:4.2f} K, tau = {r.tau:.2f}, "
          f"Ic(20 mK) = {r.Ic20 * 1e6:.2f} uA")

# --- the Ti/Al/Au junction -----
rec = ab.RECIPES[1]
sj = ab.ShortJunction(rec)
sj.calibrate(T0=0.02)                # match the measured Ic(20 mK)
print(f"\n{rec.label}: calibrated scale = {sj.scale:.3f}, "
      f"channels Nch = {sj.Nch}")
Ic20 = sj.Ic(0.02)
print(f"Ic(20 mK) after calibration: {Ic20 * 1e6:.3f} uA "
      f"(measured {rec.Ic20 * 1e6:.2f})")
assert abs(Ic20 - rec.Ic20) / rec.Ic20 < 1e-9

# Ic falls with temperature (BCS gap + thermal occupation)
for T in (0.10, 0.30, 0.50):
    print(f"   Ic({T:.2f} K) = {sj.Ic(T) * 1e6:.3f} uA")
assert sj.Ic(0.5) < sj.Ic(0.1)

# --- the occupation-noise temperature bound -----
T, tauA, t_int = 0.15, 1e-6, 1e-3    # K, s (occupation lag), s
achieved, bound, sums = sj.temperature_bound(T, tauA, t_int)
print(f"\nAndreev thermometry at T = {T} K, tauA = {tauA * 1e6:.0f} us, "
      f"t = {t_int * 1e3:.0f} ms:")
print(f"   achieved dT = {achieved * 1e6:.2f} uK")
print(f"   bound      dT = {bound * 1e6:.2f} uK "
      f"(Cauchy-Schwarz over channels)")
print(f"   Andreev heat capacity C_A = {sums['C_A'] / ab.KB:.1f} kB")
# uniform-transparency short junctions SATURATE the bound
assert abs(achieved - bound) / bound < 1e-6
print("bound saturated exactly, as the ensemble structure demands")
print("OK: lesson 5.1 complete")

```

Verified output:

```

absnoise version: 0.6.0
6 cited junction recipes:
Ta/Ti/Au      Tc* = 0.57 K, tau = 0.30, Ic(20 mK) = 1.08 uA
Ti/Al/Au      Tc* = 0.75 K, tau = 0.78, Ic(20 mK) = 2.11 uA
Ti/Al(thin)   Tc* = 0.99 K, tau = 0.53, Ic(20 mK) = 2.11 uA
Ti/Al(thick)  Tc* = 1.17 K, tau = 0.42, Ic(20 mK) = 3.17 uA
Ti/Nb/NbN     Tc* = 2.50 K, tau = 0.58, Ic(20 mK) = 0.99 uA
MoRe          Tc* = 7.40 K, tau = 0.27, Ic(20 mK) = 3.13 uA

Ti/Al/Au: calibrated scale = 0.450, channels Nch = 318
Ic(20 mK) after calibration: 2.110 uA (measured 2.11)
Ic(0.10 K) = 2.110 uA
Ic(0.30 K) = 1.889 uA
Ic(0.50 K) = 1.219 uA

Andreev thermometry at T = 0.15 K, tauA = 1 us, t = 1 ms:
achieved dT = 912.75 uK
bound      dT = 912.75 uK (Cauchy-Schwarz over channels)
Andreev heat capacity C_A = 54.0 kB

```

bound saturated exactly, as the ensemble structure demands
OK: lesson 5.1 complete

The six recipes are real, measured contact stacks with their Table-I parameters. Calibration pins the model to the measured $I_c(20\text{ mK})$ exactly, and the temperature dependence then follows from BCS + occupations with no further knobs. The punchline is the last block: the achieved temperature resolution *equals* the Cauchy-Schwarz bound $\sqrt{(2k_B T^2 \tau_A / C_A t)}$ to 10^{-6} relative — uniform-transparency short junctions saturate the bound exactly, which is a structural theorem of the ensemble, and the package reproduces it as arithmetic.

Lesson 5.2 — Telegraph noise and the full device budget

```
"""Module 5, Lesson 5.2 -- Telegraph noise and full detector budgets
(absnoise).

Occupation noise is telegraph noise: a Monte-Carlo trace of switching
channels must show the Lorentzian PSD the master equation predicts.
`SensorBudget` then assembles the device-level numbers an experiment
is designed against: thermometry resolution, resonator frequency
noise, and matched-filter calorimetric energy resolution -- checked
against its own closed-form anchor.
"""
import numpy as np

import absnoise as ab

rec = ab.RECIPES[1]                                # Ti/Al/Au
T = 0.15                                           # K
tauA = 1e-6                                       # occupation correlation time (s)

# --- telegraph Monte Carlo vs the Lorentzian plateau -----
rng = np.random.default_rng(0)
f_occ = 0.25                                     # channel occupation factor
dt, n_steps, n_ch = 2e-8, 200000, 40
x = ab.telegraph_traces(f_occ, tauA, dt, n_steps, n_ch, rng)
freqs, psd = ab.psd_single_sided(x.sum(axis=1).astype(float), dt)
# analytic single-sided plateau of n_ch two-state channels:
#  $S(0) = n_{ch} * 4 \text{ var } \tau_A, \text{ var} = f(1-f)$ 
S0_analytic = n_ch * 4.0 * f_occ * (1 - f_occ) * tauA
plateau = psd[(freqs > 1e3) & (freqs < 3e4)].mean()
print(f"telegraph PSD plateau: {plateau:.3e} "
      f"(analytic {S0_analytic:.3e})")
assert abs(plateau - S0_analytic) / S0_analytic < 0.25 # statistics-limited

# --- the device budget -----
b = ab.SensorBudget(rec)
Ce = b.Ce(T)
print(f"\nSensorBudget for {rec.label} at T = {T} K:")
print(f"  electron heat capacity Ce   : {Ce / ab.KB:.1f} kB")
print(f"  thermal time tau_th           : {b.tau_th(T) * 1e9:.2f} ns")
print(f"  Josephson inductance L_J      : {b.LJ(T) * 1e9:.3f} nH")
print(f"  resonator participation       : {b.participation(T):.4f}")
```

```

t_int = 1e-3
dT_a, _ = b.dT_andreev(T, tauA, t_int)
dT_p = b.dT_phonon(T, t_int)
print(f" dT (Andreev, 1 ms)           : {dT_a * 1e6:.1f} uK")
print(f" dT (phonon TFN, 1 ms)       : {dT_p * 1e6:.1f} uK")

# --- calorimetry: numeric matched filter vs closed form -----
sE = b.energy_resolution(T, tauA)
sE_analytic = b.energy_resolution_analytic_A(T, tauA)
eV = 1.602176634e-19
print(f" energy resolution (full)      : {sE / eV * 1e3:.2f} meV")
print(f" Andreev-only closed form      : {sE_analytic / eV * 1e3:.2f} meV")
# the full budget can only be worse than the single-noise closed form
assert sE >= 0.9 * sE_analytic
print("OK: lesson 5.2 complete")

```

Verified output:

```

telegraph PSD plateau: 2.935e-05 (analytic 3.000e-05)

SensorBudget for Ti/Al/Au at T = 0.15 K:
  electron heat capacity Ce   : 4.1 kB
  thermal time tau_th        : 1.17 ns
  Josephson inductance L_J   : 0.212 nH
  resonator participation    : 0.0960
  dT (Andreev, 1 ms)         : 2480.4 uK
  dT (phonon TFN, 1 ms)      : 113.1 uK
  energy resolution (full)    : 1.19 meV
  Andreev-only closed form    : 1.15 meV
OK: lesson 5.2 complete

```

The Monte-Carlo telegraph PSD lands on the analytic Lorentzian plateau $4 \cdot n_{\text{ch}} \cdot f(1-f) \cdot \tau_A$ within statistics (2.2% here). The budget object then gives the numbers a designer actually negotiates: heat capacity in units of k_B (4.1 k_B — these devices are *small*), the nanosecond thermal time, inductive participation, both thermometry noise floors, and a matched-filter calorimetric energy resolution of ~ 1.2 meV checked against its Andreev-only closed form. When the full numeric budget and the closed-form anchor disagree by more than the extra noise channels explain, you have found a bug — theirs or yours.

Lesson 5.3 — Fit your own junction from $I_c(T)$

```

"""Module 5, Lesson 5.3 -- Fit your own junction from a measured I_c(T)
curve (absnoise v0.6).

```

```

Your junction is not in the shipped recipe table. The standard
characterization measurement -- critical current versus temperature --
determines (I_c0, T_c*, tau), and `fit_ic_curve` fits the package's own
short-junction ensemble to it, with honest uncertainties. The fitted
recipe then re-enters every budget in the package.
"""

```

```

import numpy as np

```

```

import absnoise as ab

# --- "measure" an Ic(T) curve -----
# We synthesize data from the model with realistic noise, so the
# generating parameters are known and the fit can be judged.
Ic0_true, Tc_true, tau_true = 2.0e-6, 0.75, 0.78
T_data = np.linspace(0.05, 0.65, 12)
sigma_Ic = 0.01 * Ic0_true # 1 percent noise
rng = np.random.default_rng(5)
Ic_data = (ab.ic_model(T_data, Ic0_true, Tc_true, tau_true)
            + rng.normal(0.0, sigma_Ic, T_data.size))

# --- fit -----
fit = ab.fit_ic_curve(T_data, Ic_data, sigma_Ic=sigma_Ic)
print("fitted junction parameters (generating values in brackets):")
print(f" Ic0 = {fit.Ic0 * 1e6:.3f} +/- {fit.sigma['Ic0'] * 1e6:.3f} uA "
      f"[{Ic0_true * 1e6:.2f}]")
print(f" Tc = {fit.Tc:.4f} +/- {fit.sigma['Tc']:.4f} K [{Tc_true}]")
print(f" tau = {fit.tau:.4f} +/- {fit.sigma['tau']:.4f} [{tau_true}]")
print(f" chi2/dof = {fit.chi2 / fit.chi2_dof:.2f} (near 1: model and "
      f"stated noise consistent)")
assert abs(fit.Ic0 - Ic0_true) < 3 * fit.sigma['Ic0']
assert abs(fit.Tc - Tc_true) < 3 * fit.sigma['Tc']
assert abs(fit.tau - tau_true) < 3 * fit.sigma['tau']

# --- package the fit as a Recipe and re-enter the pipeline -----
# geometry and gating are YOUR measured values; an Ic(T) curve does
# not determine them, and to_recipe does not pretend it does.
rec = fit.to_recipe(name="my Ti/Al stack", label="mine", xi=5e-6,
                   L=0.2e-6, W=2e-6, Vbg=30.0, Rn=50.0)
sj = ab.ShortJunction(rec)
sj.calibrate()
achieved, bound, _ = sj.temperature_bound(0.15, 1e-6, 1e-3)
print(f"\nthermometry bound of the FITTED junction at 150 mK: "
      f"dT = {bound * 1e6:.0f} uK")
assert np.isfinite(bound) and bound > 0

# --- what plateau-only data cannot tell you -----
import warnings
T_low = np.linspace(0.02, 0.10, 8) # never leaves plateau
y_low = ab.ic_model(T_low, Ic0_true, Tc_true, tau_true)
with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")
    ab.fit_ic_curve(T_low, y_low, sigma_Ic=1e-9)
    msgs = [str(x.message) for x in w]
print(f"plateau-only data -> warning issued: "
      f"{any('weakly determined' in m for m in msgs)}")
assert any("weakly determined" in m for m in msgs)
print("OK: lesson 5.3 complete")

```

Verified output:

```

fitted junction parameters (generating values in brackets):
Ic0 = 1.987 +/- 0.011 uA [2.00]

```

```
Tc = 0.7455 +/- 0.0053 K    [0.75]
tau = 0.7358 +/- 0.0365     [0.78]
chi2/dof = 0.86 (near 1: model and stated noise consistent)
```

```
thermometry bound of the FITTED junction at 150 mK: dT = 936 uK
plateau-only data -> warning issued: True
OK: lesson 5.3 complete
```

The fit recovers (I_{c0} , T_c , τ) within stated uncertainties, with $\chi^2/\text{dof} = 0.86$ confirming the noise model, and `to_recipe` drops the result into the same budget machinery as the shipped recipes — your junction becomes a first-class citizen. Two honesty features to notice: geometry fields are *your* measured inputs (an $I_c(T)$ curve does not determine them, and the method does not pretend to), and plateau-only data trigger an explicit “weakly determined” warning instead of a confident wrong answer. τ is identified by the *shape* of the suppression — transparent junctions keep their supercurrent to higher T/T_c — so data that never leave the plateau contain almost no τ information, and the package says so.

5.4 Pitfalls

Two readout channels exist — current (“ I ”, at the CPR maximum) and inductance (“ L ”, near $\varphi=0$) — and their responsivities differ; keep `which=` consistent across a budget. `telegraph_traces` asserts your `dt` resolves the switching rates ($dt/\tau_A < 0.12$) rather than silently aliasing. The two-temperature click dynamics require measured phonon parameters and raise without them.

5.5 Exercises

5.1 Reproduce the bound-saturation check for all six recipes at $T/T_c = 0.2$ and rank the stacks by thermometry resolution at fixed τ_A and t .

5.2 From the fitted junction of Lesson 5.3, compute the matched-filter energy resolution and compare with the best shipped recipe.

5.3 Fit $I_c(T)$ with deliberately underestimated σ_{Ic} and watch χ^2/dof blow up — then with overestimated σ_{Ic} and watch the parameter errors inflate. Calibrate your instincts for both failure modes.

5.6 Further reading

Jung et al., Phys. Rev. Applied 26, 014078 (2026) / arXiv:2503.06850. Lee et al., Nature 586, 42 (2020). Beenakker, PRL 67, 3836 (1991) (short-junction Andreev levels).

Part III — Quantum Light and Metrology

Module 6 — cavsqueeze: Spin Squeezing You Can Design Against

Package: cavsqueeze 1.11.0 — pip install cavsqueeze — github.com/TaN-MM-Org/cavsqueeze — DOI [10.5281/zenodo.22278035](https://doi.org/10.5281/zenodo.22278035)

Methodological basis: T. M. Mahim, M. M. Rahman, A. S. M. Mohsin, “*Synchronization sets the coherence and the squeezing limit of a spin ensemble in a cavity*” (companion code: github.com/Tanvir-Mahmud-Mahim/yb-cawo4-cavity-squeezing). Squeezing conventions per Kitagawa & Ueda, PRA 47, 5138 (1993) and Wineland et al., PRA 46, R6797 (1992).

6.0 Why this package exists

Cavity-mediated one-axis twisting can squeeze the projection noise of a clock-transition spin ensemble below the standard quantum limit — but real ensembles have inhomogeneous detunings, collective decay, thermal photons and coupling spreads that mean-field treatments miss. cavsqueeze evolves the class-resolved second-order cumulant equations (cross-checked in its test suite against exact QuTiP/PIQS references and discrete truncated Wigner), ships the *exact* unitary Kitagawa-Ueda benchmark, projects the results onto clock and magnetometer sensitivity, and — v1.11 — estimates squeezing from measured tomography counts.

6.1 Conventions

from_hz builds cavity parameters from laboratory Hz (single-spin coupling g , cavity linewidth κ , detuning Δ , plus temperature and T_2); homogeneous(N) is the single-class ensemble. The Wineland parameter $\xi_2^R < 1$ is the metrological criterion; metrological_gain_db is $-10 \log_{10}(\xi_2^R)$.

Lesson 6.1 — Twist an ensemble, project a clock

```
"""Module 6, Lesson 6.1 -- Cavity-mediated spin squeezing and its
metrology projections (cavsqueeze).
```

```
A dispersively coupled cavity mediates one-axis twisting of a spin
ensemble. The class-resolved cumulant solver evolves the collective
moments; the metrology module turns them into the numbers a clock is
designed against: Wineland parameter, phase sensitivity, and the
projection-noise-limited Allan deviation.
```

```
"""
```

```
import numpy as np
```

```
import cavsqueeze as cs
```

```
from cavsqueeze import (clock_allan_deviation, css_x, from_hz,
                        homogeneous, metrological_gain_db,
                        squeezing_after, squeezing_parameters)
```

```

from cavsqueeze.cumulant import Rates

print(f"cavsqueeze version: {cs.__version__}")

# --- coherent-spin-state baseline: the standard quantum limit -----
N = 1e8
ens = homogeneous(N)
p = from_hz(g_hz=1e6 / np.sqrt(N), kappa_hz=1e4, Delta_hz=30e6,
            T=0.02, T2=0.15)
rt = Rates.from_params(p, ens)
m0 = squeezing_parameters(css_x(rt.M), rt.n)
print(f"CSS baseline: xi2_R = {m0['xi2_R']:.4f}, contrast = "
      f"{m0['contrast']:.4f}, dphi = {m0['dphi']:.3e} "
      f"(SQL 1/sqrt(N) = {1 / np.sqrt(N):.3e})")
assert np.isclose(m0["xi2_R"], 1.0) and np.isclose(m0["dphi"],
                                                    1 / np.sqrt(N))

# --- twist, with an echo, and read the squeezing -----
res = squeezing_after(p, ens, t=5e-5, echo=True)
print(f"after t = 50 us echo twist: xi2_R = {res['xi2']:.4f}, "
      f"contrast = {res['contrast']:.4f}")
gain = metrological_gain_db(res["xi2"])
print(f"metrological gain: {gain:.2f} dB over the SQL")
assert res["xi2"] < 1.0 and gain > 0.0

# --- project onto a clock -----
# dphi of the squeezed state, then the Allan deviation of a Ramsey
# clock at nu0 = 429 THz (Sr) with T_R = 100 ms, at tau = 1 s.
dphi_sq = np.sqrt(res["var_min"]) / (res["contrast"] * N / 2.0)
for label, dphi in (("SQL", 1 / np.sqrt(N)), ("squeezed", dphi_sq)):
    sy = clock_allan_deviation(dphi, nu0=4.29e14, T_ramsey=0.1, tau=1.0)
    print(f" sigma_y(1 s), {label:8s}: {sy:.3e}")
assert dphi_sq < 1 / np.sqrt(N)
print("OK: lesson 6.1 complete")

```

Verified output:

```

cavsqueeze version: 1.11.0
CSS baseline: xi2_R = 1.0000, contrast = 1.0000, dphi = 1.000e-04 (SQL 1/sqrt(N) =
↳ 1.000e-04)
after t = 50 us echo twist: xi2_R = 0.0101, contrast = 0.9997
metrological gain: 19.94 dB over the SQL
sigma_y(1 s), SQL      : 1.173e-19
sigma_y(1 s), squeezed: 1.181e-20
OK: lesson 6.1 complete

```

The coherent-spin-state block is the calibration: $\xi^2_R = 1$ and $d\phi = 1/\sqrt{N}$ to the digit — the solver reproduces the SQL before you trust it past the SQL. A 50 μs echo twist of 10^8 spins then delivers $\xi^2_R = 0.0101$ at 0.9997 contrast — 19.9 dB — and the Allan-deviation projection turns that into the number a clock team actually negotiates: $\sigma_y(1\text{ s})$ drops from 1.17×10^{-19} to 1.18×10^{-20} . (These are idealized-parameter numbers, not a claim about any laboratory clock; the point is the workflow from solver moments to σ_y .)

Lesson 6.2 — The exact benchmark and the $N^{-2/3}$ law

```
"""Module 6, Lesson 6.2 -- The exact one-axis-twisting benchmark
(cavsqueeze).

`oat_closed_form` gives the exact unitary Kitagawa-Ueda moments for N
spins -- the decoherence-free benchmark every dissipative solver is
judged against. We reproduce the textbook  $N^{-2/3}$  scaling of the
optimal squeezing and check the closed form's own internal identity
 $\xi_{2,R} = \xi_{2,S} (N/2)^2 / \langle J_x \rangle^2$ .
"""
import numpy as np

from cavsqueeze import metrological_gain_db, oat_closed_form

print(" N      |  $\xi_{2,S}(\text{opt})$  | gain (dB)")
opt = {}
for N in (100, 1000, 10000):
    mus = np.linspace(0.2, 8.0, 400) * N ** (-2.0 / 3.0)
    best = min((oat_closed_form(N, mu) for mu in mus),
               key=lambda o: o[" $\xi_{2,S}$ "])
    opt[N] = best
    print(f" {N:5d} | {best[' $\xi_{2,S}$ ']:.5f} | "
          f" {metrological_gain_db(best[' $\xi_{2,R}$ ']):.2f}")

# Kitagawa-Ueda: optimal  $\xi_{2,S}$  scales as  $N^{-2/3}$  (up to a constant)
r1 = opt[1000][" $\xi_{2,S}$ "] / opt[100][" $\xi_{2,S}$ "]
r2 = opt[10000][" $\xi_{2,S}$ "] / opt[1000][" $\xi_{2,S}$ "]
print(f"  $\xi_{2,S}$  ratio per decade of N: {r1:.3f}, {r2:.3f} "
      f" ( $N^{-2/3}$  predicts {10 ** (-2 / 3):.3f})")
assert abs(r1 - 10 ** (-2 / 3)) < 0.05
assert abs(r2 - 10 ** (-2 / 3)) < 0.02

# internal identity of the closed form
o = oat_closed_form(200, 0.03)
lhs = o[" $\xi_{2,R}$ "]
rhs = o[" $\xi_{2,S}$ "] * (200 / 2.0) ** 2 / o[" $J_x$ "] ** 2
print(f"  $\xi_{2,R}$  identity: {lhs:.6f} == {rhs:.6f}")
assert abs(lhs - rhs) < 1e-12
print("OK: lesson 6.2 complete")
```

Verified output:

```
 N      |  $\xi_{2,S}(\text{opt})$  | gain (dB)
  100 | 0.04794      | 11.91
 1000 | 0.01042      | 19.20
10000 | 0.00224      | 26.21
 $\xi_{2,S}$  ratio per decade of N: 0.217, 0.215  ( $N^{-2/3}$  predicts 0.215)
 $\xi_{2,R}$  identity: 0.096808 == 0.096808
OK: lesson 6.2 complete
```

The Kitagawa-Ueda closed form (every line of it pinned against brute-force exact evolution in the package's tests) reproduces the textbook scaling: optimal $\xi_{2,S}$ falls per decade of N by 0.217 and 0.215 against the $N^{-2/3}$ prediction of 0.215. When you wire up a dissipative

simulation, run it against this benchmark first at small N — decoherence should *degrade* it, and any improvement over the unitary optimum is a bug, not a discovery.

Lesson 6.3 — Squeezing from measured counts

```

"""Module 6, Lesson 6.3 -- Squeezing from measured count data
(cavsqueeze v1.11).

Every other module predicts; this one estimates from an experiment.
We simulate the standard Ramsey tomography record -- per-angle shots
of the population difference J_z -- from the EXACT one-axis-twisting
covariance, then hand only the counts to `estimate_squeezing` and
compare its answer against the closed form it has never seen.
"""

import numpy as np

from cavsqueeze import (estimate_squeezing, metrological_gain_db,
                        oat_closed_form)

# --- ground truth: exact OAT state of N = 100 spins -----
N, mu = 100, 0.05
oat = oat_closed_form(N, mu)
contrast_true = 2.0 * oat["Jx"] / N
print(f"truth: Vmin = {oat['Vmin']:.4f}, xi2_R = {oat['xi2_R']:.4f}, "
      f"contrast = {contrast_true:.4f}")

# transverse covariance in the (y, z) tomography plane
a0 = oat["alpha_min"]
R = np.array([[np.cos(a0), -np.sin(a0)], [np.sin(a0), np.cos(a0)]])
cov = R @ np.diag([oat["Vmin"], oat["Vmax"]]) @ R.T

def v_theta(th):
    u = np.array([np.cos(th), np.sin(th)])
    return float(u @ cov @ u)

# --- "measure": M shots of J_z at each tomography angle -----
rng = np.random.default_rng(0)
angles = np.linspace(0.0, np.pi, 12, endpoint=False)
M = 4000
shots = [rng.normal(0.0, np.sqrt(v_theta(t)), M) for t in angles]

# --- estimate from the counts alone -----
est = estimate_squeezing(angles, shots, N, contrast=contrast_true)
print(f"estimated: V_min = {est.var_min:.4f} +/- {est.var_min_sigma:.4f}"
      f" (truth {oat['Vmin']:.4f})")
print(f"estimated: xi2_R = {est.xi2_R:.4f} +/- {est.xi2_R_sigma:.4f}"
      f" (truth {oat['xi2_R']:.4f})")
print(f"gain: {metrological_gain_db(est.xi2_R):.2f} dB "
      f"(truth {metrological_gain_db(oat['xi2_R']):.2f})")
d = (est.theta_min - a0) % np.pi
print(f"dip angle error (mod pi): {min(d, np.pi - d):.4f} rad")

```

```

assert abs(est.var_min - oat["Vmin"]) < 4 * est.var_min_sigma
assert abs(est.xi2_R - oat["xi2_R"]) < 4 * est.xi2_R_sigma
assert est.xi2_R < 1.0

# --- detection noise: subtract it or answer the wrong question -----
vdet = 2.0 # known detector variance
shots_noisy = [s + rng.normal(0.0, np.sqrt(vdet), M) for s in shots]
raw = estimate_squeezing(angles, shots_noisy, N, contrast=contrast_true)
corr = estimate_squeezing(angles, shots_noisy, N, contrast=contrast_true,
                        detection_variance=vdet)
print(f"with detector noise: raw xi2_R = {raw.xi2_R:.3f}, "
      f"corrected = {corr.xi2_R:.3f} (truth {oat['xi2_R']:.3f})")
assert abs(corr.xi2_R - oat["xi2_R"]) < 5 * corr.xi2_R_sigma
assert raw.xi2_R > corr.xi2_R # noise masquerades as anti-squeezing
print("OK: lesson 6.3 complete")

```

Verified output:

```

truth: Vmin = 3.1277, xi2_R = 0.1331, contrast = 0.9695
estimated: V_min = 3.3215 +/- 0.0990 (truth 3.1277)
estimated: xi2_R = 0.1413 +/- 0.0042 (truth 0.1331)
gain: 8.50 dB (truth 8.76)
dip angle error (mod pi): 0.0041 rad
with detector noise: raw xi2_R = 0.226, corrected = 0.140 (truth 0.133)
OK: lesson 6.3 complete

```

The estimator never sees the model — only per-angle shot records — and lands within uncertainties of the closed form it was generated from, dip angle included. The fit leans on exact structure: $V(\theta) = c + a \cdot \cos 2\theta + b \cdot \sin 2\theta$ is the rotation law of *any* 2×2 covariance (not a Gaussian assumption), so the tomography fit is linear, and $V_{\min}$ is an eigenvalue identity. The detection-noise block teaches the field's classic trap: unsubtracted detector variance masquerades as anti-squeezing (0.226 vs the true 0.133); the opt-in subtraction recovers 0.140, and over-subtraction into unphysical territory *raises* rather than reporting infinite squeezing.

6.4 Pitfalls

The one Gaussian assumption in `estimate_squeezing` is the sample-variance error bar $2s^4/(M-1)$ — stated in the docstring; heavy-tailed detection noise inflates real error bars beyond it. Tomography needs ≥ 3 distinct quadratures mod π or the fit refuses. For weighted (inhomogeneously coupled) ensembles the coherent-state normalization is S_2/S_1^2 , handled by `wineland_xi2` — do not divide by N by hand.

6.5 Exercises

6.1 Sweep the twist time in Lesson 6.1 with `squeezing_trace` and locate the optimum; compare against `optimal_squeezing`'s automated search.

6.2 Reproduce the OAT benchmark with the cumulant solver at $N = 100$ with all decoherence off and quantify the cumulant truncation error against `oat_closed_form`.

6.3 In Lesson 6.3, halve M and double the number of angles at fixed total shots; which allocation gives a smaller ξ^2_R error bar? Check against the exact WLS covariance.

6.6 Further reading

Kitagawa & Ueda, PRA 47, 5138 (1993). Wineland et al., PRA 46, R6797 (1992). Ma, Wang, Sun, Nori, Phys. Rep. 509, 89 (2011). Itano et al., PRA 47, 3554 (1993) (projection-noise clock statistics).

Module 7 — sqzcomb: Squeezed Light from Kerr Microcombs

Package: sqzcomb 0.8.0 — `pip install sqzcomb` — github.com/TaN-MM-Org/sqzcomb — DOI [10.5281/zenodo.22015375](https://doi.org/10.5281/zenodo.22015375)

Methodological basis: T. M. Mahim, M. M. Rahman, A. S. M. Mohsin, “Overcoming the 3 dB squeezing extraction limit in silicon carbide microcombs with a photonic molecule,” *Optics Express* 34(18), 34822 (2026), doi:10.1364/OE.612248 (open access; code: github.com/Tanvir-Mahmud-Mahim/sic-molecule-squeezer).

7.0 Why this package exists

A Kerr microresonator below comb threshold is a parametric squeezer, but the squeezing you *detect* is throttled by extraction: a single critically coupled ring cannot deliver more than 3 dB. The paper’s answer is a photonic molecule — a second, strongly out-coupled ring that routes the squeezing off the chip — and sqzcomb is the general engine behind it: Lugiato-Lefever steady states (flat, solitons, crystals), linearized Bogoliubov fluctuations with stability guards, detected spectra through input-output theory, detection/loss budgets, Gaussian entanglement measures, and — v0.8 — the entanglement of the *output* fields a detector actually sees.

7.1 Conventions

Dimensionless LLE units (decay rate 1); vacuum quadrature variance is 1/2. `squeezing_db(v)` is $10 \cdot \log_{10}(v/0.5)$, negative for squeezed. Drift matrices act on the doubled (modes, conjugates) basis; every spectral function refuses an unstable drift matrix — and, since v0.7, distinguishes a genuinely unstable one from the marginal Goldstone mode of a soliton (`allow_marginal=True` when you know what you are doing).

Lesson 7.1 — Steady states and the DPO anchor

```
"""Module 7, Lesson 7.1 -- Kerr-comb steady states and squeezing spectra (sqzcomb).
```

```
The Lugiato-Lefever equation's flat steady states obey an exact cubic; the linearized fluctuations around any steady state give detected squeezing spectra through input-output theory. The degenerate parametric oscillator is the closed-form anchor -- including the famous 3 dB extraction limit at critical coupling.
"""
```

```
import numpy as np
```

```
import sqzcomb
```

```
from sqzcomb import (fluctuation_matrix, homogeneous_steady_states,
                     output_quadrature_variance, single_mode_parametric,
```

```

squeezing_db)

print(f"sqzcomb version: {sqzcomb.__version__}")

# --- LLE flat states satisfy the exact cubic -----
F, alpha = 1.1, 0.5 # pump amplitude, detuning
rhos = homogeneous_steady_states(F, alpha) # |psi|^2 roots
print(f"flat-state intensities for F={F}, alpha={alpha}: "
      f"{np.round(rhos, 6)}")
for rho in rhos:
    resid = rho * (1.0 + (alpha - rho) ** 2) - F ** 2
    assert abs(resid) < 1e-9 # the exact cubic
print("every root satisfies rho[1 + (alpha - rho)^2] = F^2 exactly")

# --- DPO closed form: S(Omega) = 1 - 4 eta mu / ((1+mu)^2 + Omega^2)
mu, eta = 0.9, 1.0
M = single_mode_parametric(mu)
print("\nDPO squeezing spectrum (phi = pi/2 quadrature, eta = 1):")
print(" Omega | package variance | closed form")
for om in (0.0, 0.7, 2.5):
    v = output_quadrature_variance(M, eta, om, mode_index=0, n_modes=1,
                                   phi=np.pi / 2)
    v_exact = 0.5 * (1 - 4 * eta * mu / ((1 + mu) ** 2 + om ** 2))
    print(f" {om:.3f} | {v:.6f} | {v_exact:.6f}")
    assert abs(v - v_exact) < 1e-10

# --- the 3 dB extraction limit at critical coupling -----
M99 = single_mode_parametric(0.9999)
v_crit = output_quadrature_variance(M99, 0.5, 0.0, 0, 1, phi=np.pi / 2)
print(f"\ncritical coupling (eta = 1/2), mu -> 1: "
      f"{squeezing_db(v_crit):.3f} dB (limit: -3.01 dB)")
assert -3.011 < squeezing_db(v_crit) < -2.99

# --- an LLE comb state's fluctuation matrix -----
rho0 = float(rhos[0])
psi_s = np.sqrt(rho0) * np.ones(8, dtype=complex)
Mc, modes = fluctuation_matrix(psi_s, alpha, dispersion=(0.0, 0.0, 0.15))
print(f"comb fluctuation matrix: {Mc.shape} over modes {modes}")
stable = np.max(np.linalg.eigvals(Mc).real) < 0.0
print(f"flat lower-branch state stable: {stable}")
assert stable
print("OK: lesson 7.1 complete")

```

Verified output:

```

sqzcomb version: 0.8.0
flat-state intensities for F=1.1, alpha=0.5: [0.981935]
every root satisfies rho[1 + (alpha - rho)^2] = F^2 exactly

DPO squeezing spectrum (phi = pi/2 quadrature, eta = 1):
Omega | package variance | closed form
0.0 | 0.001385 | 0.001385
0.7 | 0.060976 | 0.060976
2.5 | 0.317444 | 0.317444

```

```
critical coupling (eta = 1/2), mu -> 1: -3.010 dB (limit: -3.01 dB)
comb fluctuation matrix: (16, 16) over modes [ 0  1  2  3 -4 -3 -2 -1]
flat lower-branch state stable: True
OK: lesson 7.1 complete
```

The flat states satisfy the exact LLE cubic $\rho[1+(\alpha-\rho)^2] = F^2$ to 10^{-9} — an algebraic identity, your first check on any parameter set. The degenerate-parametric-oscillator spectrum then matches its closed form to 10^{-10} at every (μ, η, Ω) tested, and the celebrated extraction limit emerges from the machinery itself: at critical coupling ($\eta = 1/2$), even pumped to 99.99% of threshold, the detected level saturates at -3.010 dB. That number is the reason Module 7 exists.

Lesson 7.2 — The photonic molecule and the loss budget

```
"""Module 7, Lesson 7.2 -- The photonic molecule and honest detection
budgets (sqzcomb).
```

```
A single Kerr ring cannot be critically coupled without paying the
3 dB extraction penalty; coupling it to an auxiliary "extraction ring"
(a photonic molecule) routes the squeezing out. The detection module
then applies the loss and dark-noise model every real receiver adds --
and inverts it into a loss budget.
"""
```

```
import numpy as np
```

```
from sqzcomb import (detected_squeezing_db, molecule_threshold,
                     output_variance_ports, photonic_molecule,
                     required_efficiency_db, squeezing_db)
```

```
# --- molecule: threshold and detected squeezing -----
J, gamma = np.sqrt(3.0), 1.0 # ring-ring coupling, aux loss
thr = molecule_threshold(J, gamma)
print(f"molecule threshold at  $J^2/\text{gamma} = \{J ** 2 / \text{gamma} : .1f\}$ : "
      f" $\mu_{\text{thr}} = \{\text{thr} : .4f\}$  (single ring: 1)")
```

```
mu = 0.98 * thr # pump at 98% of threshold
```

```
M, gams = photonic_molecule(mu, J, gamma=gamma)
# read the squeezing through the AUXILIARY ring's bus (port mode 1);
# the -iJ hop rotates the squeezed quadrature a quarter turn, so the
# aux port squeezes at phi = 0 where the Kerr ring squeezes at pi/2.
v_aux = output_variance_ports(M, gams, eta=1.0, port_mode=1,
                              omega=0.0, phi=0.0)
```

```
print(f"detected variance through the auxiliary port:  $\{v_{\text{aux}} : .5f\}$  "
      f" $\rightarrow \{\text{squeezing\_db}(v_{\text{aux}}) : .2f\}$  dB")
```

```
# the critically coupled single ring saturates at 3.01 dB; the
# molecule's dedicated extraction ring goes past that wall
assert squeezing_db(v_aux) < -4.0
```

```
# --- what the photodiodes actually report -----
```

```
print("\nreceiver chain (loss eta, dark clearance):")
```

```
for eta in (1.0, 0.9, 0.7, 0.5):
    db = detected_squeezing_db(v_aux, efficiency=eta)
```

```

    print(f" eta = {eta:.1f}: {db:+.2f} dB")
# loss can only degrade squeezing, monotonically
dbs = [detected_squeezing_db(v_aux, efficiency=e)
        for e in (1.0, 0.9, 0.7, 0.5)]
assert all(a < b for a, b in zip(dbs, dbs[1:]))

# --- invert the model: the loss budget -----
target_db, source_db = -3.0, squeezing_db(v_aux)
eta_min = required_efficiency_db(target_db, source_db)
print(f"\nto deliver {target_db} dB from a {source_db:.2f} dB source, "
      f"total efficiency must exceed {eta_min:.3f}")
db_at_min = detected_squeezing_db(v_aux, efficiency=eta_min)
print(f"check: at eta = {eta_min:.3f} the detected level is "
      f"{db_at_min:.3f} dB")
assert abs(db_at_min - target_db) < 1e-9
print("OK: lesson 7.2 complete")

```

Verified output:

```

molecule threshold at  $J^2/\gamma = 3.0$ :  $\mu_{thr} = 2.0000$  (single ring: 1)
detected variance through the auxiliary port: 0.16893 -> -4.71 dB

receiver chain (loss eta, dark clearance):
eta = 1.0: -4.71 dB
eta = 0.9: -3.94 dB
eta = 0.7: -2.70 dB
eta = 0.5: -1.75 dB

to deliver -3.0 dB from a -4.71 dB source, total efficiency must exceed 0.753
check: at eta = 0.753 the detected level is -3.000 dB
OK: lesson 7.2 complete

```

Coupling the Kerr ring to a dedicated extraction ring ($J^2/\gamma = 3$) raises the threshold to $\mu_{thr} = 2$ and routes the squeezing out through the auxiliary bus: -4.71 dB detected at full efficiency in this configuration — past the 3 dB wall a critically coupled single ring cannot cross. (Note the quarter-turn: the $-iJ$ hop rotates the squeezed quadrature, so the auxiliary port squeezes at $\varphi = 0$ where the ring squeezes at $\pi/2$ — a classic sign-convention trap the lesson makes explicit.) The receiver chain then shows loss eating squeezing monotonically, and `required_efficiency_db` inverts the model into the one-line loss budget: delivering -3 dB from this source needs total efficiency ≥ 0.753 , verified by round trip.

Lesson 7.3 — Entanglement, intracavity and detected

```

"""Module 7, Lesson 7.3 -- Entanglement, intracavity and detected
(sqzcomb v0.8).

```

```

The two-mode squeezed vacuum's log-negativity is  $E_N = 2r$  exactly --
the intracavity anchor. The v0.8 output-field toolbox then computes
what a detector SEES: the entanglement of the propagating output
modes of a nondegenerate parametric process, with the twin-beam
closed form  $E_N = -\ln(2 V_{EPR\_min})$  as its anchor and detection loss
that can only degrade it.
"""

```

```

import numpy as np

from sqzcomb import (logarithmic_negativity, output_covariance_xxpp,
                     output_entanglement, output_entanglement_spectrum,
                     ppt_symplectic_eigenvalue)

def tmsv_cov(r, hbar=2.0):
    """Two-mode squeezed vacuum covariance, xxpp ordering."""
    c, s = np.cosh(2 * r), np.sinh(2 * r)
    unit = hbar / 2.0
    sig = unit * np.array([[c, s, 0, 0],
                           [s, c, 0, 0],
                           [0, 0, c, -s],
                           [0, 0, -s, c]])

    return sig

# --- intracavity anchor:  $E_N(\text{TMSV}) = 2r$  exactly -----
for r in (0.3, 0.8, 1.5):
    EN = logarithmic_negativity(tmsv_cov(r))
    print(f"TMSV  $r = \{r\}$ :  $E_N = \{EN:.6f\}$  (exact  $2r = \{2 * r\}$ )")
    assert abs(EN - 2 * r) < 1e-12

# --- output fields of a nondegenerate parametric process -----
# modes ( $a_1, a_2$ ) coupled by a pump:  $da_1 = -a_1 + \mu a_2^*$ , symmetric.
mu, eta = 0.6, 0.8

def twin_drift(mu):
    A = -np.eye(2, dtype=complex)
    B = np.array([[0.0, mu], [mu, 0.0]], dtype=complex)
    return np.block([[A, B], [np.conj(B), np.conj(A)]])

M = twin_drift(mu)
sig_out = output_covariance_xxpp(M, eta, 0.0, n_modes=2)
print(f"\noutput covariance shape: {sig_out.shape} (xxpp)")
EN_out = output_entanglement(M, eta, 0.0, 0, 1, 2)
# twin-beam closed form:  $E_N = -\ln(2 V_{\text{EPR\_min}})$ , from the same cov
x1, x2 = sig_out[0, 0], sig_out[1, 1]
x12 = sig_out[0, 1]
p1, p2, p12 = sig_out[2, 2], sig_out[3, 3], sig_out[2, 3]
V_minus = 0.5 * (x1 + x2 - 2 * x12) # Var[(x1 - x2)/sqrt(2)]
V_plus = 0.5 * (p1 + p2 + 2 * p12) # Var[(p1 + p2)/sqrt(2)]
V_epr = min(V_minus, V_plus)
print(f"detected  $E_N$  at carrier: {EN_out:.6f}")
print(f" $-\ln(2 V_{\text{EPR\_min}})$  : {-np.log(2 * V_epr):.6f}")
assert abs(EN_out - (-np.log(2 * V_epr))) < 1e-9

# --- the entanglement spectrum, and loss monotonicity -----
omegas = np.array([0.0, 0.5, 1.0, 2.0, 4.0])
spec = output_entanglement_spectrum(M, eta, omegas, 0, 1, 2)
print("E_N(Omega):", np.round(spec, 4))

```

```

assert np.all(np.diff(spec) <= 1e-12)      # decays away from carrier
spec_lossy = output_entanglement_spectrum(M, eta, omegas, 0, 1, 2,
                                           detection_efficiency=0.6)
print("with 60% detection:", np.round(spec_lossy, 4))
assert np.all(spec_lossy <= spec + 1e-12)  # loss never creates E_N

# PPT sanity: an unpumped (vacuum) output is separable
EN_vac = output_entanglement(twin_drift(0.0), eta, 0.0, 0, 1, 2)
print(f"unpumped output E_N: {EN_vac:.2e} (separable: 0)")
assert EN_vac < 1e-12
print("OK: lesson 7.3 complete")

```

Verified output:

```

TMSV r = 0.3: E_N = 0.600000 (exact 2r = 0.6)
TMSV r = 0.8: E_N = 1.600000 (exact 2r = 1.6)
TMSV r = 1.5: E_N = 3.000000 (exact 2r = 3.0)

output covariance shape: (4, 4) (xxpp)
detected E_N at carrier: 1.386294
-ln(2 V_EPR_min)      : 1.386294
E_N(Omega): [1.3863 1.1497 0.7751 0.3463 0.1092]
with 60% detection: [0.5978 0.5276 0.391 0.1931 0.0641]
unpumped output E_N: 2.22e-16 (separable: 0)
OK: lesson 7.3 complete

```

The two-mode squeezed vacuum anchor $E_N = 2r$ comes out exact to 10^{-12} for three squeezing strengths — the intracavity entanglement toolbox (Simon PPT criterion, log-negativity per Vidal & Werner) is calibrated. The v0.8 output toolbox then evaluates the twin-beam entanglement a detector sees: E_N at the carrier equals $-\ln(2 \cdot V_{\text{EPR_min}})$ computed independently from the same output covariance (1.386294 both ways), the spectrum decays away from the carrier, 60% detection efficiency degrades but never creates entanglement, and the unpumped output is exactly separable. Every statement is an inequality or identity you can re-run.

7.4 Pitfalls

Above-threshold drift matrices are refused — linearized spectra around an unstable state are meaningless; find the correct steady state first (`newton_state`). The molecule's port quadrature is rotated a quarter turn from the ring's. Thermal inputs: a passive cavity emits exactly $(2\bar{n}+1)/2$ at every frequency — use that closed form to sanity-check any thermal configuration before trusting a pumped one.

7.5 Exercises

7.1 Map detected squeezing versus (η, μ) for the single ring and verify the 3 dB ceiling holds along the entire critical-coupling line as $\mu \rightarrow 1$.

7.2 Scan the molecule's J^2/γ from 0.5 to 5 at fixed 98%-of-threshold pumping and find the extraction optimum for this configuration; compare against reading the *Kerr ring's* own port.

7.3 Compute `output_entanglement_spectrum` with unequal per-arm detection efficiencies and show the asymmetric loss still never increases E_N .

7.6 Further reading

Mahim, Rahman, Mohsin, Opt. Express 34, 34822 (2026). Lugiato & Lefever, PRL 58, 2209 (1987). Vidal & Werner, PRA 65, 032314 (2002). Simon, PRL 84, 2726 (2000). Weedbrook et al., Rev. Mod. Phys. 84, 621 (2012).

Part IV — Closing the Loop with Fabrication

Module 8 — fabtwin: Learn the Fab, Then Beat Its Tail

Package: fabtwin 0.2.0 — `pip install fabtwin` (JAX twin: `pip install 'fabtwin[twin]'`) — github.com/TaN-MM-Org/fabtwin — DOI [10.5281/zenodo.22697049](https://doi.org/10.5281/zenodo.22697049)

Methodological basis: T. M. Mahim, M. N. Islam, M. M. Rahman, A. S. M. Mohsin, “FabGAN-ID: Learning the Fabrication Process for Yield-Aware Inverse Design of Multilayer Photonic Sensor Filters,” IEEE Sensors Journal (2026); companion repository github.com/Tanvir-Mahmud-Mahim/Learned-generative-process-twins-for-yield-aware-inverse-design-of-multilayer-photonic-sensor (benchmark archived at DOI [10.5281/zenodo.21315793](https://doi.org/10.5281/zenodo.21315793)).

8.0 Why this package exists

Real deposition errors are systematic, layer-correlated, design-conditional, skewed, and heavy-tailed — and manufacturing yield is decided precisely in the tail that hand-specified Gaussian tolerance models represent worst. The FabGAN-ID idea: *learn what cannot be simulated; differentiate what can*. Learn the fabrication process from historical (recipe, outcome) traces, then robustify designs by ascending the yield-deciding lower-tail statistic (CVaR) pathwise through the learned twin **and** the exact transfer-matrix physics at once. fabtwin is the generalized library: any layer count, any design box, any cited material system, any linear-in-T merit — with the entire differentiable loop in pure NumPy, because the discrete adjoint of the transfer recursion is derived by hand. The optional [twin] extra adds the conditional WGAN-GP twin in JAX, whose autodiff gradients are asserted equal to the hand adjoint to 10^{-12} — two independent derivations of one physics, the package’s core cross-validation.

8.1 Conventions

Wavelengths and thicknesses in μm . Layer index $n(\lambda)$ enters as a per-layer array or as $n_0 \cdot S(\lambda)$ with a shared or per-layer dispersion shape S ; cited Sellmeier materials refuse evaluation outside their fitted validity range. Merits are linear in T: $J = w \cdot T + c$, built by `notch_weights / bandpass_weights` or by hand. The hand adjoint covers normal incidence and lossless designs — the FabGAN-ID regime; the oblique/absorbing *forward* solver exists, and its gradients are deliberately not faked.

Lesson 8.1 — Exact optics and adjoint design

```
"""Module 8, Lesson 8.1 -- Exact thin-film optics and adjoint inverse
design (fabtwin).
```

```
The characteristic-matrix solver is pinned to closed forms (here: the
quarter-wave antireflection transform and  $R + T = 1$ ), the hand-derived
adjoint gradient is checked against finite differences, and the
probe-seeded design engine recovers the analytic AR optimum.
"""
```

```

import numpy as np

import fabtwin as ft

print(f"fabtwin version: {ft.__version__}")

# --- closed-form anchors -----
lam0 = 0.55                                # um
ns = 1.52                                  # glass substrate
# quarter-wave AR coating:  $n = \sqrt{ns}$ ,  $t = \text{lam0} / (4 n) \rightarrow R = 0$ 
n_ar = np.sqrt(ns)
t_ar = lam0 / (4.0 * n_ar)
lam = np.array([lam0])
T = ft.transmittance(lam, np.array([t_ar]), np.array([n_ar])), 1.0, ns)
R = ft.reflectance(lam, np.array([t_ar]), np.array([n_ar]), 1.0, ns)
R0, T0 = float(R[0]), float(T[0])
print(f"quarter-wave AR at {lam0} um: R = {R0:.2e} (exact 0), "
      f"R + T = {R0 + T0:.12f}")
assert R0 < 1e-12 and abs(R0 + T0 - 1.0) < 1e-12

# energy conservation on a random lossless stack
rng = np.random.default_rng(0)
tr = rng.uniform(0.03, 0.10, 6)
nr = rng.uniform(1.5, 2.3, 6)
lam_g = np.linspace(0.45, 0.65, 21)
Tr = ft.transmittance(lam_g, tr, nr[:, None] * np.ones((1, 21)), 1.0, ns)
Rr = ft.reflectance(lam_g, tr, nr[:, None] * np.ones((1, 21)), 1.0, ns)
print(f"random 6-layer stack: max |R + T - 1| = "
      f"{np.abs(Rr + Tr - 1).max():.2e}")
assert np.abs(Rr + Tr - 1).max() < 1e-12

# --- the hand adjoint vs central finite differences -----
S = np.ones((1, lam_g.size))                # dispersionless for the check
w, c0 = ft.bandpass_weights(lam_g, 0.53, 0.57, 0.02)
t0 = rng.uniform(0.04, 0.09, 4)
n0 = rng.uniform(1.6, 2.2, 4)
Sfull = np.ones((4, lam_g.size))
J0, gt, gn = ft.merit_and_grad(lam_g, t0, n0, Sfull, w, c0, n_sub=ns)
h = 1e-6
fd = np.empty(4)
for i in range(4):
    tp, tm = t0.copy(), t0.copy()
    tp[i] += h
    tm[i] -= h
    Jp = ft.merit_and_grad(lam_g, tp, n0, Sfull, w, c0, n_sub=ns)[0]
    Jm = ft.merit_and_grad(lam_g, tm, n0, Sfull, w, c0, n_sub=ns)[0]
    fd[i] = (Jp - Jm) / (2 * h)
err = np.abs(gt - fd).max() / np.abs(fd).max()
print(f"adjoint dJ/dt vs finite differences: max rel err {err:.2e}")
assert err < 1e-6

# --- inverse design recovers the analytic AR optimum -----
box = ft.DesignBox(1, 0.02, 0.20, 1.05, 2.4)
# merit = T(lam0): weights [1], constant 0 -- maximize transmission

```

```

t_opt, n_opt, J_opt, hist = ft.inverse_design(
    np.array([lam0]), np.ones((1, 1)), np.array([1.0]), 0.0, box,
    n_sub=ns, seed=1)
to, no = float(t_opt[0]), float(n_opt[0])
print(f"designed AR layer: t = {to:.6f} um (analytic {t_ar:.6f}), "
      f"n = {no:.6f} (analytic {n_ar:.6f})")
assert abs(to - t_ar) < 1e-3 and abs(no - n_ar) < 1e-3
print("OK: lesson 8.1 complete")

```

Verified output:

```

fabtwin version: 0.2.0
quarter-wave AR at 0.55 um: R = 1.15e-33 (exact 0), R + T = 1.000000000000
random 6-layer stack: max |R + T - 1| = 6.66e-16
adjoint dJ/dt vs finite differences: max rel err 5.35e-10
designed AR layer: t = 0.111570 um (analytic 0.111527), n = 1.233101 (analytic
↳ 1.232883)
OK: lesson 8.1 complete

```

Three anchors in one lesson: the quarter-wave AR transform gives $R = 10^{-33}$ at the design wavelength with $R+T = 1$ to 12 digits; a random six-layer stack conserves energy to 10^{-16} across the grid; the hand-derived adjoint matches central finite differences at 5×10^{-10} — the FD-limited accuracy, meaning the adjoint is *more* accurate than the check. The probe-seeded design engine then recovers the analytic AR optimum ($t = \lambda/4n$, $n = \sqrt{n_{\text{sub}}}$) to about 2×10^{-4} — optimizer-tolerance, honestly reported, not machine precision.

Lesson 8.2 — Learn the process, ascend the tail

```

"""Module 8, Lesson 8.2 -- Process twins and pathwise CVaR
robustification (fabtwin).

```

```

The six-mechanism virtual deposition process generates historical
(recipe, outcome) traces; a Gaussian twin learns the error
distribution; and `robustify` ascends the yield-deciding lower-tail
statistic (CVaR) pathwise through the twin and the exact physics at
once. The robustified design must beat the nominal design in the tail
it optimizes -- under the TRUE process, which the twin never saw.
"""

```

```
import numpy as np
```

```
import fabtwin as ft
```

```
rng = np.random.default_rng(0)
```

```

# --- platform: 8-layer SiNx notch filter on silica -----
lam = np.linspace(0.45, 0.65, 41)
S = ft.dispersion_shape(lam, 0.550)          # Si3N4 shape (Luke 2015)
nsub = ft.SiO2_MALITSON1965.n(lam)          # substrate (Malitson 1965)
w, c0 = ft.notch_weights(lam, 0.532, 0.015, 0.030)
box = ft.DesignBox(8, 0.020, 0.120, 1.6, 2.4)

```

```
# nominal design
```

```
t0, n0, J0, _ = ft.inverse_design(lam, S, w, c0, box, n_sub=nsub,
```

```

n_iter=30, seed=0)
print(f"nominal design merit: {J0:.4f}")

# --- historical traces and the Gaussian twin -----
recipes = box.sample(rng, 150)
rt, rn, ftd, fnd = ft.PAPER_PROCESS.trace_dataset(*recipes, 2, rng)
x = ft.errors_from_traces(rt, rn, ftd, fnd)
print(f"trace dataset: {rt.shape[0]} runs -> error vectors {x.shape}")
twin = ft.GaussianTwin(x)

# --- robustify: CVaR ascent through twin + physics -----
alpha = 0.10
t_r, n_r, hist = ft.robustify(lam, t0, n0, S, w, c0, twin, box,
                              alpha=alpha, K=64, steps=40, lr=2e-3,
                              seed=1, n_sub=nsub)
obj = [h[1] for h in hist] # (step, objective) pairs
print(f"CVaR objective: start {obj[0]:.4f} -> end {obj[-1]:.4f}")
assert obj[-1] >= obj[0] - 1e-9 # ascent does not degrade

# --- score BOTH designs under the true process -----
def sampler(a, b, K):
    return ft.PAPER_PROCESS.ensemble(a, b, K, rng)

rep_nom = ft.evaluate_under_process(sampler, t0, n0, lam, S, w, c0,
                                    K=3000, n_sub=nsub, alpha=alpha)
rep_rob = ft.evaluate_under_process(sampler, t_r, n_r, lam, S, w, c0,
                                    K=3000, n_sub=nsub, alpha=alpha)
print("          nominal    robustified")
for key in ("mean", "CVaR"):
    print(f"    {key:6s}      {rep_nom[key]:.4f}    {rep_rob[key]:.4f}")
print(f"tail (CVaR_{alpha}) improvement under the TRUE process: "
      f"{rep_rob['CVaR'] - rep_nom['CVaR']:+.4f}")
assert rep_rob["CVaR"] > rep_nom["CVaR"] # the point of the loop
print("OK: lesson 8.2 complete")

```

Verified output:

```

nominal design merit: 0.7756
trace dataset: 300 runs -> error vectors (300, 16)
CVaR objective: start 0.7026 -> end 0.7258
          nominal    robustified
mean      0.7259    0.7622
CVaR      0.6509    0.7214
tail (CVaR_0.1) improvement under the TRUE process: +0.0704
OK: lesson 8.2 complete

```

This is the whole FabGAN-ID loop in one page. Historical traces from the six-mechanism reference process train a Gaussian twin; robustify ascends $\text{CVaR}_{0.1}$ pathwise through twin and physics ($0.7026 \rightarrow 0.7258$ on the twin's own objective); and the verdict comes from the only referee that matters — the *true* process, which the twin never saw: the robustified design's worst-decile mean improves by $+0.070$ in merit while the mean also rises. On real problems the mean sometimes pays for the tail; here it did not, and the honest experiment is to check both, every time.

Lesson 8.3 — Bring your own fabrication data

```
"""Module 8, Lesson 8.3 -- Bring your own fabrication data (fabtwi  
v0.2).
```

```
The twins never see a simulator: they consume (recipe, outcome)  
traces, which is exactly what a monitored deposition tool logs. This  
lesson runs the complete real-data workflow: traces to CSV and back  
(exact round trip), structural validation with plausibility flags,  
a held-out twin-fidelity report, and the honest scope statement that  
goes with it.  
"""
```

```
import os  
import tempfile
```

```
import numpy as np
```

```
import fabtwi as ft
```

```
rng = np.random.default_rng(7)
```

```
# --- a "fab log": traces in the documented CSV contract -----  
box = ft.DesignBox(8, 0.020, 0.120, 1.6, 2.4)  
recipes = box.sample(rng, 40)  
rt, rn, ftd, fnd = ft.PAPER_PROCESS.trace_dataset(*recipes, 2, rng)
```

```
path = os.path.join(tempfile.mkdtemp(), "fab_traces.csv")  
ft.save_traces_csv(path, rt, rn, ftd, fnd)  
rt2, rn2, ft2, fn2 = ft.load_traces_csv(path)  
print(f"CSV round trip exact: "  
      f"{np.array_equal(rt, rt2) and np.array_equal(fn2, fnd)}")  
assert np.array_equal(rt, rt2) and np.array_equal(ftd, ft2)
```

```
# --- validation: structure raises, plausibility is reported -----  
rep = ft.validate_traces(rt2, rn2, ft2, fn2)  
print(f"validated: {rep['n_runs']} runs x {rep['n_layers']} layers, "  
      f"{len(rep['flags'])} plausibility flags")  
mixed = ft2.copy()  
mixed[3, 2] *= 1000.0 # a nm-vs-um unit mix-up  
rep2 = ft.validate_traces(rt2, rn2, mixed, fn2)  
print(f"after injecting a unit mix-up: {len(rep2['flags'])} flag(s), "  
      f"first = (run {rep2['flags'][0][0]}, layer {rep2['flags'][0][1]}, "  
      f"{rep2['flags'][0][2]})")  
assert len(rep2["flags"]) >= 1 # flagged, not silently dropped
```

```
# --- held-out twin fidelity -----  
lam = np.linspace(0.45, 0.65, 41)  
S = ft.dispersion_shape(lam, 0.550)  
nsub = ft.SIO2_MALITSON1965.n(lam)  
w, c0 = ft.notch_weights(lam, 0.532, 0.015, 0.030)
```

```
x = ft.errors_from_traces(rt2, rn2, ft2, fn2)  
x_fit, x_hold = x[:, :2], x[:, 1:2] # fit on half, score on rest  
twin = ft.GaussianTwin(x_fit)
```

```

designs = [(rt2[0], rn2[0]), (rt2[5], rn2[5])]
report = ft.twin_fidelity_report(
    twin.sample_errors(np.random.default_rng(1), 400), x_hold,
    designs, lam, S, w, c0, n_sub=nsub, alpha=0.1)
print("held-out fidelity report (paper Table I(A) statistic types):")
print(f"  moments : d_mean {report['moments']['d_mean']:.4f}, "
      f"d_corr {report['moments']['d_corr']:.3f}")
print(f"  induced : |dP10| {report['induced']['d_P']:.4f}, "
      f"|dCVaR| {report['induced']['d_CVaR']:.4f}, "
      f"W1 {report['induced']['W1']:.4f}")

# a deliberately broken twin must score strictly worse
broken = ft.GaussianTwin(x_fit + 0.05)
rep_b = ft.twin_fidelity_report(
    broken.sample_errors(np.random.default_rng(1), 400), x_hold,
    designs, lam, S, w, c0, n_sub=nsub, alpha=0.1)
print(f"  broken twin W1: {rep_b['induced']['W1']:.4f} "
      f"(> fitted twin's {report['induced']['W1']:.4f})")
assert rep_b["induced"]["W1"] > report["induced"]["W1"]
assert rep_b["moments"]["d_mean"] > report["moments"]["d_mean"]

print("\nHONEST SCOPE: a held-out split certifies the twin against your "
      "fab's RECORDED behavior; it cannot certify error patterns the "
      "tool has never logged, and in production the twin needs periodic "
      "retraining as the tool drifts.")
print("OK: lesson 8.3 complete")

```

Verified output:

```

CSV round trip exact: True
validated: 80 runs x 8 layers, 0 plausibility flags
after injecting a unit mix-up: 1 flag(s), first = (run 3, layer 2, rel_t)
held-out fidelity report (paper Table I(A) statistic types):
  moments : d_mean 0.0197, d_corr 3.027
  induced : |dP10| 0.0009, |dCVaR| 0.0068, W1 0.0046
  broken twin W1: 0.0315 (> fitted twin's 0.0046)

HONEST SCOPE: a held-out split certifies the twin against your fab's RECORDED
↳ behavior; it cannot certify error patterns the tool has never logged, and in
↳ production the twin needs periodic retraining as the tool drifts.
OK: lesson 8.3 complete

```

The v0.2 real-data pathway is the package's answer to the paper's own "essential next step": the twins consume (recipe, outcome) traces — exactly what a monitored deposition tool logs — through a documented CSV contract with an exact round trip; validation *raises* on structural corruption but only *flags* plausibility findings (the injected 1000× unit mix-up is reported, not silently dropped — a lab's heavy tail is precisely what the twin is for). With no hidden oracle to draw truth from, `twin_fidelity_report` scores the twin the honest way — held-out split, the paper's Table I(A) statistic types, through the exact solver — and a deliberately broken twin scores strictly worse on every axis. The scope statement printed at the end is not boilerplate; it is the epistemics of the method: a held-out split certifies *recorded* behavior, and production twins need retraining as the tool drifts.

8.4 Pitfalls

Layer thicknesses in μm , not nm — the trace-file contract’s column names (`t_recipe_um`) exist to prevent exactly the unit mix-up the validator flags. Gain (negative extinction) is refused by convention. The correction-policy layer (the paper’s Stage 3) is deliberately absent — the paper scopes it as exploratory — but all the pathwise machinery a policy trainer needs is exported. Neural forward surrogates are absent *by result*: the paper’s protocol study found the exact differentiable solver strictly dominates them in this regime.

8.5 Exercises

8.1 Build a two-material $\text{Si}_3\text{N}_4/\text{SiO}_2$ quarter-wave mirror with per-layer dispersion shapes and verify the $(\text{HL})^p$ reflectance against the textbook admittance closed form $R = ((1-Y)/(1+Y))^2$ for $p = 1, 3$.

8.2 Rerun Lesson 8.2 with the process’s particulate rate `p_flake` doubled (via `data-classes.replace` on `PAPER_PROCESS`) and measure how much of the CVaR gain survives a heavier tail.

8.3 Fit the Gaussian twin on 20 traces instead of 300 and watch the held-out fidelity report catch the degradation before you waste a robustification run on it.

8.4 (With `fabtwin[twin]`) Train the WGAN-GP twin on the same traces and compare its held-out `W1` against the Gaussian twin’s — then read the docstring note reproducing the paper’s finding on when physics-in-the-loop calibration helps.

8.6 Further reading

Mahim, Islam, Rahman, Mohsin, IEEE Sensors J. (2026). Macleod, *Thin-Film Optical Filters* (4th ed.). Byrnes, arXiv:1603.02720 (the independent `tmm` reference the solver is cross-validated against). Rockafellar & Uryasev, J. Risk 2, 21 (2000) (CVaR). Luke et al., Opt. Lett. 40, 4823 (2015); Malitson, JOSA 55, 1205 (1965) (the cited materials).

Capstone Projects

Each capstone crosses at least two modules; all are doable with the shipped packages alone.

C1 — From Hamiltonian to detector. Use hamop to compute the transmission of a defected ribbon; treat the transmission dip as a sensor merit and use fabtwin’s risk module (cvar, evaluate_under_process with your own sampler) to quantify its yield under hopping disorder.

C2 — A calibrated Raman study, end to end. Simulate a full strain+doping map with ramansep’s forward model, add realistic spectra with the Lorentzian generator, run the complete pipeline (fit → invert → multimode check → Bayesian map), and write the uncertainty section of the “paper” from the propagated errors alone.

C3 — The emitter factory. Combine sparq’s SPRT with the Bayesian verdict posterior: triage a simulated wafer of 200 sites under a photon budget, and compare total acquisition time and error rates against fixed-dwell analysis at equal confidence.

C4 — Squeezing, delivered. Chain cavsqueeze and sqzcomb conceptually: compute the spin-projection Allan gain a given detected optical squeezing level would enable, then use required_efficiency_db to budget the optical chain that delivers it.

C5 — Design for the fab you have. Log “traces” from a deliberately drifted DepositionProcess (change a mechanism mid-dataset), watch twin_fidelity_report degrade on the held-out recent half, retrain, and quantify the yield cost of stale twins.

Appendix A — Version and Citation Table

Package	Version	PyPI	Concept DOI (cite all versions)
hamop	0.6.0	pypi.org/project/hamop/	10.5281/zenodo.22311381
kpenvelope	0.7.0	pypi.org/project/kpenvelope/	10.5281/zenodo.22015269
ramansep	0.7.0	pypi.org/project/ramansep/	10.5281/zenodo.22014913
sparq-triage	0.5.0	pypi.org/project/sparq-triage/	10.5281/zenodo.22278041
absnoise	0.6.0	pypi.org/project/absnoise/	10.5281/zenodo.22048608
cavsqueeze	1.11.0	pypi.org/project/cavsqueeze/	10.5281/zenodo.22278035
sqzcomb	0.8.0	pypi.org/project/sqzcomb/	10.5281/zenodo.22015375
fabtwinn	0.2.0	pypi.org/project/fabtwinn/	10.5281/zenodo.22697049

Every package: Apache-2.0, CITATION.cff in the repository, issues welcome at the respective GitHub tracker (github.com/TaN-MM-Org). Usage questions are treated as documentation bugs when a docstring left a unit or sign convention unclear.

Appendix B — The Code Bundle

The companion zip contains one directory per module (m1_hamop ... m8_fabtwine), each with the lesson scripts exactly as printed in this book and a `.out.txt` capture of the verified output next to each script. Run any lesson as `python lesson_X_Y_name.py`; every script is self-checking (it *asserts* its own anchors) and prints OK on success. If an assertion ever fails on your machine with the pinned versions, that is reportable — file an issue with the script name and your platform.

Colophon

This course was prepared against the released PyPI artifacts of all eight packages (not development checkouts); every code listing was executed and its output captured verbatim on 2026-09-11. The author thanks the readers who run the assertions.

© 2026 Tanvir Mahmud Mahim. *The course text is for purchasers' personal use; the code snippets in the lessons are trivial API usage and may be reused freely in your research. The packages themselves are and remain Apache-2.0 open source — this course is how the project is supported, not a paywall on the software.*